



POLITECNICO DI MILANO

MSc in High Performance Computing Engineering
Course in Advanced Methods for Scientific Computing

A parallel finite element solver for parabolic equations on quantum graphs

AUTHOR

Yosef Ben Chedly (279568)

SUPERVISOR

Prof. Paola F. Antonietti

COURSE PROFESSOR

Prof. Luca Formaggia

Abstract

This work develops and validates a finite element framework for the numerical solution of differential problems on metric graphs, with particular emphasis on parabolic equations. The implementation is first verified on elliptic, parabolic and spectral benchmark problems with known reference solutions. The framework is then applied to a Fisher–Kolmogorov diffusion–reaction model for the propagation of misfolded proteins in the brain connectome.

A performance study is subsequently carried out on increasingly refined connectome discretisations. The analysis shows how finite element matrix reordering and parallel programming techniques can be combined to reduce the computational cost of the solver. Exploiting the natural separation between connection-interior unknowns and graph vertices leads to a condensed formulation in which the interior degrees of freedom can be processed independently, while the global problem is reduced to the original graph vertices. At this level, the formulation recovers a nodal description comparable to those adopted in the reference network models, while retaining the possibility of resolving the dynamics along each connection through the underlying finite element discretisation. The resulting independent per-connection computations are then parallelized using OpenMP, MPI and a hybrid approach. This combination of algebraic reorganisation and parallel execution substantially accelerates the scientific workload while exposing the natural support of both shared- and distributed-memory parallelism.

Keywords: metric graphs; quantum graphs; finite element method; parabolic equations; Fisher–Kolmogorov equation; brain connectome; high-performance computing; OpenMP; MPI.

Contents

1	Analytical and numerical aspects of quantum graphs	5
1.1	Basic definitions	5
1.2	Finite element discretization	7
2	Parabolic problems on metric graphs	9
2.1	Problem formulation	9
2.2	Matrix formulation	10
2.3	Fully discrete system	11
3	Numerical Benchmarks	13
3.1	Elliptic Test Cases	13
3.1.1	Test E1: constant function	13
3.1.2	Test E2: linear function	14
3.1.3	Test E3: function of a single variable	15
3.2	Parabolic Test Cases	17
3.2.1	Test 1: radially decaying solution	17
3.3	Elliptic eigenvalue problems	18
3.3.1	Test 1: four-pointed star graph	18
3.3.2	Test 2: graphene-like graph	19
3.3.3	Test 3: binary tree graph	21
3.4	Parabolic eigenvalue problems	24
3.4.1	Test 4: heat equation on the graphene-like graph	24
3.4.2	Test 5: heat equation on the binary tree graph	24
3.4.3	Laplace matrix spectral comparison	24
4	Spreading of neurodegenerative diseases within the brain's connectome	29
4.1	One-dimensional sensitivity analysis	29
4.2	The brain connectome as a metric graph	31
4.3	Network model and metric-graph formulation	34
4.4	Comparison with the network model	36
4.5	What controls the regional separation	37
4.5.1	Mass lumping as a stabilization technique	40
4.6	Tau seeding and expected progression	41
4.7	Deterministic model with regional reaction rates	42
5	Performance analysis	44
5.1	Sequential baseline	44
5.2	Matrix ordering and the sequential solver	45
5.3	Static condensation	46
5.4	Parallel implementation	48
5.4.1	OpenMP	48
5.4.2	MPI	48
5.5	Parallel performance	49
6	Conclusions and future work	51
7	C++ implementation	52
7.1	Repository reference	52
7.2	Dependencies	52
7.3	Computational approach to differential problems	52
7.4	Input data	52
7.5	Setting coefficients	53
7.6	Assembly procedures	53
7.7	Solvers	53
7.8	Exporting results	53
7.9	Result postprocessing	53
7.10	Example of code usage	53

7.11 Parallel code excerpts	54
A Properties of the Combinatorial Laplacian	56
B Derivation of the weak and algebraic formulations	57
B.1 Problem formulation	57
B.2 Weak formulation	57
B.3 Derivation of the variational formulation	58
B.4 Galerkin formulation	59
B.5 Matrix formulation	60
B.6 Fully discrete system	60

Introduction

Many physical and biological processes evolve on structures that are naturally described as networks. In a purely combinatorial graph, however, connections only describe which vertices are linked, without resolving the physical behaviour occurring between them. Metric graphs enrich this representation by associating each connection with a one-dimensional domain, so that differential equations can be posed and solved directly along the edges. Quantum graphs further equip this structure with differential operators and appropriate coupling conditions at the vertices. This provides a compact framework for modelling phenomena in which both network topology and continuous dynamics along the connections are relevant.

The first objective of this work is to develop a finite element framework for differential problems posed on such graphs. Each connection is discretised with standard one-dimensional piecewise linear finite elements, while continuity and Neumann–Kirchhoff conditions couple the local edge problems at the graph vertices. The resulting formulation leads naturally to a global system containing both the interior finite element unknowns of the connections and the original graph vertices. The implementation is validated through a sequence of elliptic, parabolic and spectral benchmark problems, including spatial and temporal convergence tests.

The second objective is to investigate the behaviour of the framework on a realistic network-based application. For this purpose, the Fisher–Kolmogorov equation is considered on a brain connectome to describe the diffusion and local replication of misfolded proteins associated with neurodegenerative diseases. The model is used to investigate the interplay between diffusion, non-linear reaction and regional connectivity within a computationally efficient reduced-order framework.

The final objective concerns computational efficiency. Refining the connections increases the number of finite element unknowns while leaving the number of original graph vertices unchanged. This structure is exploited in the performance analysis. First, the effect of sparse matrix ordering and factorization on the sequential solver is investigated. The natural ordering of the unknowns reveals a block structure in which the interior unknowns of different connections are independent. Static condensation eliminates these interior unknowns locally, reducing the global problem to a small interface system involving only the original vertices. The full global sparse matrix is consequently avoided and most of the remaining computational work becomes independent across connections. This decomposition provides a natural basis for parallelization. Shared-memory parallelism is implemented with OpenMP by distributing connections among threads, while MPI partitions the same work among processes with separate memory spaces. A hybrid MPI-OpenMP implementation combines the two approaches.

This report is organized as follows. Chapter 1 introduces the analytical setting of quantum and metric graphs and their finite element discretization. Chapter 2 derives the formulation of parabolic problems and their fully discrete approximation. Chapter 3 validates the numerical implementation through elliptic, parabolic and spectral benchmarks. Chapter 4 applies the framework to Fisher–Kolmogorov dynamics on the brain connectome. Chapter 5 investigates the sequential and parallel performance of the solver, including matrix ordering, static condensation, OpenMP and MPI. Chapter 6 summarizes the main results and discusses possible future developments. Finally, Chapter 7 documents the C++ implementation of the framework, describing the structure of the library, the main solver components and the input-output pipeline.

1 Analytical and numerical aspects of quantum graphs

Quantum graphs have attracted significant interest as a tool for modeling phenomena involving a wide range of physical systems: from wave propagation in photonic crystals and thin waveguides to molecular dynamics and signal transmission in social, financial networks and neural architectures in the context of neurosciences. One can find a collection of papers on quantum graphs in [1]. Together with their potential interdisciplinarity, they gained popularity also because they naturally arise as reduced-order models.

Here we provide the essential theoretical background on quantum graphs needed for the computational development of this project. We emphasize that the material covered here is by no means exhaustive; a comprehensive introduction to quantum graphs can be found in [2]. Therefore, the purpose of this section is to introduce the reader to the class of mathematical models known as *quantum graphs*, first from an analytical perspective and then by describing the numerical methods used to study them.

1.1 Basic definitions

Let us begin with an intuitive definition. A *quantum graph* is a collection of one-dimensional intervals connected at their endpoints to form a network. The term “quantum” refers to the fact that the graph is equipped with a Hamiltonian operator that acts on the functions supported in these intervals, coupled by suitable boundary conditions at the vertices [3]. We already provided the main idea behind quantum graphs, but for a more rigorous formulation, we need the following Definitions:

Definition 1.1 (directed graphs). A **directed graph** is a pair of sets $\Gamma = (\mathcal{V}, \mathcal{A})$, where:

- $\mathcal{V} = \{v_i\}_{i=1}^N$ is a set of **vertices** (or nodes);
- $\mathcal{A} \subseteq \mathcal{V} \times \mathcal{V}$ is a set of ordered pairs of vertices, called **arcs**.
- Given an arc $(v_i, v_j) \in \mathcal{A}$, it is said to be outgoing from v_i and incoming to v_j . Two nodes v_i and v_j are said to be adjacent if $(v_i, v_j) \in \mathcal{A}$.

Definition 1.2 (undirected graphs). An **undirected graph** is a pair of sets $\Gamma = (\mathcal{V}, \mathcal{E})$, where:

- $\mathcal{V} = \{v_i\}_{i=1}^N$ is a set of **vertices** (or nodes);
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is a set of unordered pairs of vertices, called **edges**.
- An edge connecting two vertices v_i, v_j is written $\{v_i, v_j\}$ with $i < j$. Two nodes v_i and v_j are said to be adjacent if $\{v_i, v_j\} \in \mathcal{E}$.

Definition 1.3 (combinatorial and metric graphs). Definition 1.1 and Definition 1.2 present arcs and edges as binary relations between pairs of vertices. Two nodes are either linked together or not: we don't know anything about the “strength” of the connection. For this reason, the graphs that we presented are called **combinatorial graphs**. One can choose a measure for the length of such connections. A combinatorial graph endowed with geometry is a **metric graph**.

Now we present some useful matrices used for the description of combinatorial (and metric) graphs.

Definition 1.4 (adjacency matrix). The **adjacency matrix** of a graph Γ with N vertices is the symmetric $N \times N$ matrix $A \in \{0, 1\}^{N \times N}$, defined by

$$A_{ij} = \begin{cases} 1 & \text{if vertices } v_i, v_j \text{ are adjacent,} \\ 0 & \text{otherwise.} \end{cases}$$

Definition 1.5 (incidence matrix). The **incidence matrix** of a directed graph $\Gamma = (\mathcal{V}, \mathcal{A})$ with N vertices and M arcs is the $N \times M$ matrix E , whose column corresponding to arc (v_i, v_j) has exactly two nonzero entries: $+1$ in row i and -1 in row j . The same holds for an undirected graph with edges $\{v_i, v_j\}$.

Definition 1.6 (combinatorial Laplacian). The **combinatorial Laplacian** of Γ is the $N \times N$ symmetric positive semidefinite singular matrix

$$L_\Gamma = EE^\top.$$

For the analysis of the properties of the combinatorial Laplacian, we refer to Appendix A.

Definition 1.7 (natural coordinates). Each edge e of a (undirected) metric graph can be associated to a finite interval $(0, \ell_e) \subset \mathbb{R}$, with **natural coordinate** s_e , so that s_e gives the position of a point along edge e . Note that assigning a coordinate requires a choice of direction: we take the increasing direction of s_e to coincide with the orientation fixed when defining the incidence matrix E . For $e = \{v_i, v_j\}$, s_e will therefore increase in the direction that goes from v_i to v_j , with $i < j$. The same definition holds for natural coordinates on arcs of a directed graph, with increasing direction coinciding with the one of the arc itself.

The definitions of metric graphs and natural coordinates are very important to introduce functional spaces over Γ .

Definition 1.8 ($L^2(\Gamma)$). Let $\Gamma = (\mathcal{V}, \mathcal{E})$ be a metric graph. For each edge $e \in \mathcal{E}$, let

$$L^2(e) := \left\{ f: e \rightarrow \mathbb{R} \mid \int_e |f|^2 ds < \infty \right\}.$$

The space of square-integrable functions on Γ is defined as

$$L^2(\Gamma) := \bigoplus_{e \in \mathcal{E}} L^2(e).$$

In other terms, $f \in L^2(\Gamma)$ if and only if

$$\|f\|_{L^2(\Gamma)}^2 = \sum_{e \in \mathcal{E}} \|f\|_{L^2(e)}^2 < \infty.$$

Definition 1.9 ($H^1(\Gamma)$). For each edge $e \in \mathcal{E}$, let

$$H^1(e) := \left\{ f \in L^2(e) \mid \int_e |f'(s)|^2 ds < \infty \right\}.$$

One definition of $H^1(\Gamma)$ is:

$$H^1(\Gamma) := \left(\bigoplus_{e \in \mathcal{E}} H^1(e) \right) \cap C^0(\Gamma),$$

where $C^0(\Gamma)$ is the space of continuous functions on Γ .

In other terms, $f \in H^1(\Gamma)$ if and only if f is continuous on Γ and

$$\|f\|_{H^1(\Gamma)}^2 = \sum_{e \in \mathcal{E}} \|f\|_{H^1(e)}^2 < \infty.$$

The idea behind the definition of a quantum graph is that edges may be characterised not only by constant coefficients, but also by variable ones following physical laws, possibly described by

differential equations. In particular, vertices can interact through such laws. To formalise this, we assign to the metric graph Γ an additional structure represented by a differential operator $\mathcal{H}$, called the *Hamiltonian*. In particular:

$$\mathcal{H}u(s) := -\frac{d^2u(s)}{ds^2} + V(s)u(s), \quad (1)$$

where $V(s) \in L^\infty(\Gamma)$ is a given potential and derivatives are taken along the edge coordinate s . To pose a well-defined differential problem on Γ , we must supplement $\mathcal{H}$ with a set of vertex conditions $\mathcal{C}$.

Definition 1.10 (quantum graphs). *A quantum graph is a triple $(\Gamma, \mathcal{H}, \mathcal{C})$, where $\Gamma = (\mathcal{V}, \mathcal{E})$ is a metric graph, $\mathcal{H}$ is a differential operator acting edgewise on it and $\mathcal{C}$ is a set of vertex conditions imposed at $\mathcal{V}$.*

Examples of boundary conditions are the so called *Neumann-Kirchhoff conditions*.

The remainder of this report is devoted to the investigation of differential problems on quantum graphs. In particular, we will notice that also more complicated operators acting on Γ can be considered, as the one generating parabolic problems.

1.2 Finite element discretization

The classical finite element method can be employed to study quantum graphs. Let $\Gamma = (\mathcal{V}, \mathcal{E})$ be a metric graph, where each edge $e \in \mathcal{E}$ is identified with an interval $(0, \ell_e)$. For an edge $e = \{v_p, v_q\}$, we fix an orientation so that $s_e = 0$ corresponds to v_p and $s_e = \ell_e$ to v_q . On each edge e , we introduce a uniform partition into n_e subintervals of length $h_e = \ell_e/n_e$. This yields the nodes

$$\{s_j^e\}_{j=0}^{n_e}, \quad s_j^e = jh_e,$$

with $s_0^e = 0$ and $s_{n_e}^e = \ell_e$, corresponding to the vertices v_p and v_q , respectively. The interior nodes are $\{s_j^e\}_{j=1}^{n_e-1}$. This construction induces a new graph, which we refer to as the *extended graph*. Its vertex set consists of the original vertices $\mathcal{V}$ together with all the interior nodes introduced by the discretization, while its edge set is given by the segments connecting consecutive nodes along each original edge. On this extended graph, we construct the standard hat basis functions $\{\phi_j^e\}_{j=1}^{n_e-1}$, associated with the interior nodes of each edge, together with the functions $\{\phi_v\}_{v \in \mathcal{V}}$, related to the original vertices. All these functions are piecewise linear and globally continuous on Γ . More precisely, each function ϕ_j^e is supported on the two elements adjacent to the node s_j^e , takes the value 1 at s_j^e , and vanishes at all other nodes. Similarly, for each vertex $v \in \mathcal{V}$, the function ϕ_v takes the value 1 at v and decreases linearly to zero along each edge incident to v , vanishing at the neighboring nodes. In practice, we first construct, on each edge, the usual hat functions of the standard finite element method (yielding $n_e - 1$ degrees of freedom per edge). We then enrich this construction by adding basis functions associated with the vertices, thereby incorporating the topological structure of the graph. In this way, the classical finite element basis is naturally extended to the setting of (quantum) graphs. We define the local finite element space on each edge as

$$V_h^e := \{u_h \in H^1(e) : u_h \text{ is piecewise linear with respect to the partition}\}.$$

The global finite element space is then given by

$$V_h(\Gamma) := \bigoplus_{e \in \mathcal{E}} V_h(e)$$

which is a finite-dimensional subspace of $H^1(\Gamma)$, in which we'll search approximate solutions to the weak formulation of differential problems. Any function $u_h \in V_h(\Gamma)$ can be written as a linear combination of the basis functions:

$$u_h(s) = \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \alpha_j^e \phi_j^e(s) + \sum_{v \in \mathcal{V}} \beta_v \phi_v(s). \quad (2)$$

One observation that we can make is that it is convenient to order the vertices by following the original edge structure of the graph. More precisely, for each edge, the vertices introduced by the

discretization (i.e., the interior nodes along that edge) are numbered consecutively. After all such nodes have been listed for every edge, the original vertices of the graph are numbered last. In this way, the vertices corresponding to the subdivided edges (the subdomain nodes) appear in contiguous blocks within linear algebra structures, while the original graph vertices (the separators) are grouped together at the end of these. This is shown in the following Figure.

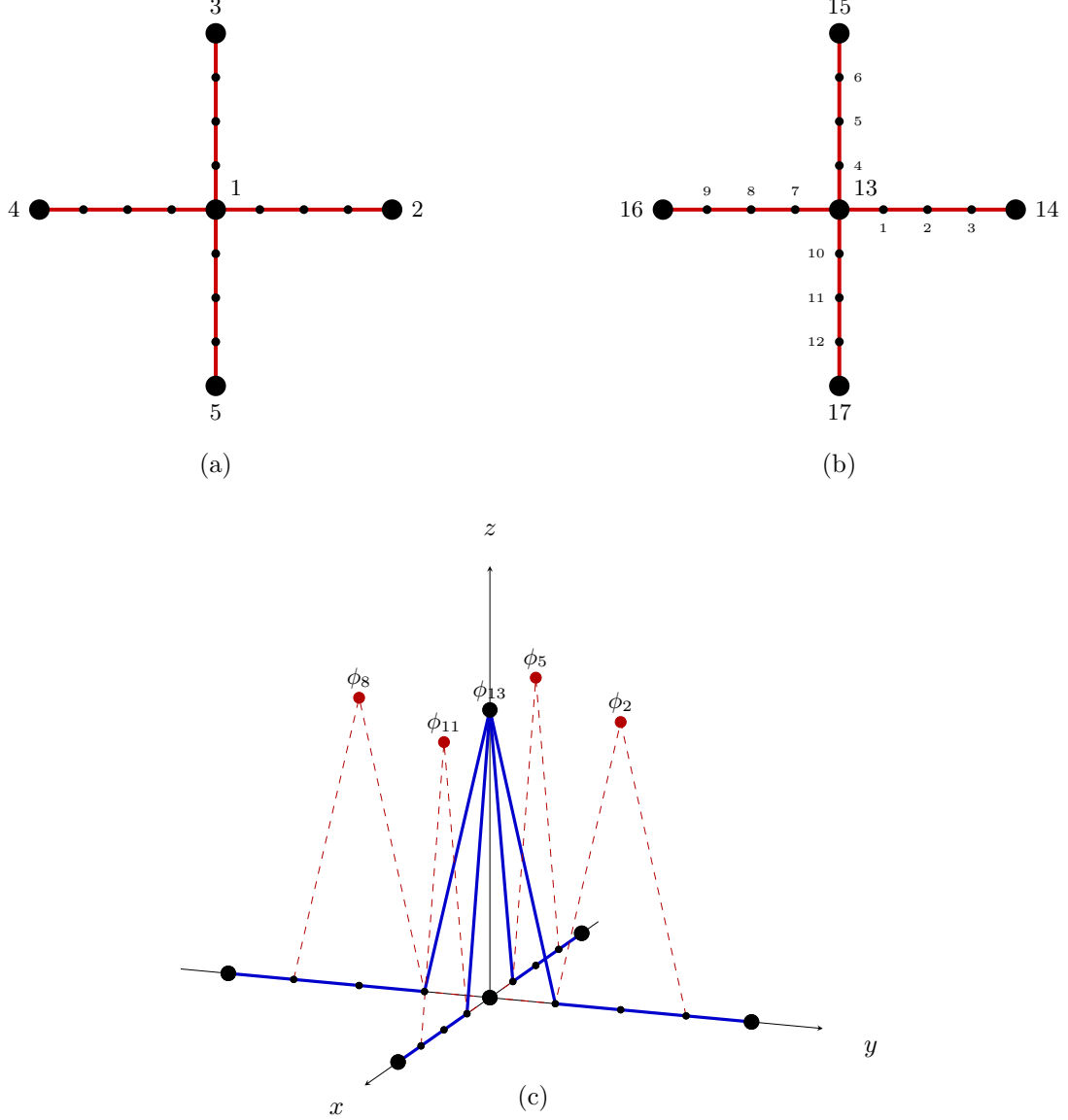


Figure. (a) A star-shaped graph $\Gamma = (\mathcal{V}, \mathcal{E})$ with five vertices ($\mathcal{V} = \{v_1, v_2, v_3, v_4, v_5\}$) and four edges ($\mathcal{E} = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_1, v_4\}, \{v_1, v_5\}\}$), equipped with a uniform mesh consisting of four elements per edge. The edges incident to the central vertex v_1 are highlighted in red. (b) The same star graph with global node numbering: interior discretisation nodes are listed first, edge by edge (right: 1–3; top: 4–6; left: 7–9; bottom: 10–12, each numbered from centre outward; this orientation coincides with the directionality chosen for the construction of the incidence matrix), followed by the true graph vertices (centre: 13; right: 14; top: 15; left: 16; bottom: 17). (c) The graph Γ together with its finite-element basis. In blue, the vertex basis function ϕ_{13} ; in dashed red, four representative hat basis functions, one per arm.

2 Parabolic problems on metric graphs

A comprehensive analysis on graph-based parabolic problems can be found in [4].

2.1 Problem formulation

We consider quantum graphs with parabolic Hamiltonians. Let $\Gamma = (\mathcal{V}, \mathcal{E})$ be a metric graph, $V \in L^\infty(\Gamma)$ and $f : \Gamma \rightarrow \mathbb{R}$ be given. Let $\mathbb{H}$ denote the elliptic Hamiltonian of the quantum graph $(\Gamma, \mathbb{H}, \mathcal{C}_{NK})$, with underlying Neumann-Kirchhoff conditions at each vertex in $\mathcal{V}$ and global continuity constraints. The parabolic Hamiltonian acts on functions defined on Γ as follows:

$$\mathbb{H} u(s, t) = \frac{\partial u(s, t)}{\partial t} + \mathcal{H} u(s, t) = \frac{\partial u(s, t)}{\partial t} - \frac{\partial^2 u(s, t)}{\partial s^2} + V(s) u(s, t),$$

where $\mathcal{H}$ is the elliptic Hamiltonian introduced with (Eq. 1).

Let us consider the differential problem

$$\begin{cases} \mathbb{H} u = f & \text{in } \Gamma \times (0, T) \\ \sum_{e \in \mathcal{E}_{v_p}} \left[\frac{\partial u_e}{\partial \mathbf{n}_e} \right]_v = 0, & \forall v \in \mathcal{V}, \forall t \in (0, T) \end{cases} \quad (3)$$

where dependence on the variables (s, t) have been omitted in the first equation.

This problem can be expressed in weak form by means of a standard procedure reported in Appendix B, yielding

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u}{\partial t} w \, ds + \sum_{e \in \mathcal{E}} \int_e \frac{\partial u}{\partial s} \frac{\partial w}{\partial s} \, ds + \sum_{e \in \mathcal{E}} \int_e V u w \, ds = \sum_{e \in \mathcal{E}} \int_e f w \, ds, \forall w \in H^1(\Gamma). \quad (4)$$

Defining

$$a(u, w) = \sum_{e \in \mathcal{E}} \int_e \left[\frac{\partial u}{\partial s} \frac{\partial w}{\partial s} + V u w \right] \, ds, \quad F(w) = \sum_{e \in \mathcal{E}} \int_e f w \, ds,$$

the weak formulation reads:

Find $u \in H^1[\Gamma \times (0, T)]$, globally continuous on Γ such that

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u}{\partial t} w \, ds + a(u, w) = F(w) \quad \forall w \in H^1(\Gamma). \quad (5)$$

The discrete version of the problem can be obtained exploiting a standard Galerkin procedure. Let $V_h \subset H^1(\Gamma)$ be the space spanned by the basis functions $\{\phi_j^e\}_{j=1}^{n_e-1}$ (interior node hat functions on each edge) and $\{\phi_v\}_{v \in \mathcal{V}}$ (vertex basis functions), as defined in SECTION 1.2. The discrete solution is written as in Eq.(2) and substituted into the weak formulation (5), whose terms are therefore restricted to the subspace V_h . We first test on an interior node hat function of an arbitrary edge. The time term gives:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_h}{\partial t} \phi_k^{e'} \, ds = \sum_{j=1}^{n_{e'}-1} \dot{\alpha}_j^{e'} \underbrace{\int_{e'} \phi_j^{e'} \phi_k^{e'} \, ds}_{P_{kj}^{e'}} + \sum_{v \in \mathcal{V}} \dot{\beta}_v \underbrace{\int_{e'} \phi_v \phi_k^{e'} \, ds}_{R_{kv}^{e'}}$$

where only the $e = e'$ term survives due to the local support of $\phi_k^{e'}$ and $R_{kv}^{e'}$ is nonzero only for the (at most two) vertices incident to e' . The stiffness term gives:

$$\begin{aligned} a(u_h, \phi_k^{e'}) &= \sum_{j=1}^{n_{e'}-1} \alpha_j^{e'} \underbrace{\int_{e'} \left[\frac{\partial \phi_j^{e'}}{\partial s} \frac{\partial \phi_k^{e'}}{\partial s} + V_{e'} \phi_j^{e'} \phi_k^{e'} \right] \, ds}_{(H_{11})_{kj}^{e'}} \\ &\quad + \sum_{v \in \mathcal{V}} \beta_v \underbrace{\int_{e'} \left[\frac{\partial \phi_v}{\partial s} \frac{\partial \phi_k^{e'}}{\partial s} + V_{e'} \phi_v \phi_k^{e'} \right] \, ds}_{(H_{12})_{kv}^{e'}}. \end{aligned}$$

The forcing term gives:

$$F(\phi_k^{e'}) = \int_{e'} f_{e'} \phi_k^{e'} ds.$$

We then test on an arbitrary vertex basis function. The time term gives:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_h}{\partial t} \phi_{v'} ds = \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \dot{\alpha}_j^e \underbrace{\int_e \phi_j^e \phi_{v'} ds}_{R_{v'j}^e} + \sum_{v \in \mathcal{V}} \dot{\beta}_v \underbrace{\sum_{e \in \mathcal{E}} \int_e \phi_v \phi_{v'} ds}_{Q_{v'v}},$$

where $Q_{vv'}$ is nonzero only when v and v' are the same. The stiffness term gives:

$$\begin{aligned} a(u_h, \phi_{v'}) &= \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \alpha_j^e \underbrace{\int_e \left[\frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + V_e \phi_j^e \phi_{v'} \right] ds}_{(H_{21})_{v'j}^e} + \\ &+ \sum_{v \in \mathcal{V}} \beta_v \underbrace{\left[\sum_{e \in \mathcal{E}} \int_e \left(\frac{\partial \phi_v}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + V_e \phi_v \phi_{v'} \right) ds \right]}_{(H_{22})_{v'v}}, \end{aligned}$$

while the forcing term

$$F(\phi_{v'}) = \sum_{e \in \mathcal{E}} \int_e f_e \phi_{v'} ds.$$

2.2 Matrix formulation

Collecting all unknowns into $\mathbf{u} = [\mathbf{u}_{\mathcal{E}}, \mathbf{u}_{\mathcal{V}}]^\top$, where $\mathbf{u}_{\mathcal{E}} = \{\{\alpha_j^e\}_{j=1}^{n_e-1}\}_{e \in \mathcal{E}}$ and $\mathbf{u}_{\mathcal{V}} = \{\beta_v\}_{v \in \mathcal{V}}$, the Galerkin system is

$$\mathbf{M} \begin{bmatrix} \dot{\mathbf{u}}_{\mathcal{E}}(t) \\ \dot{\mathbf{u}}_{\mathcal{V}}(t) \end{bmatrix} + \mathbf{H} \begin{bmatrix} \mathbf{u}_{\mathcal{E}}(t) \\ \mathbf{u}_{\mathcal{V}}(t) \end{bmatrix} = \begin{bmatrix} \mathbf{f}_{\mathcal{E}}(t) \\ \mathbf{f}_{\mathcal{V}}(t) \end{bmatrix}, \quad (6)$$

with

$$\mathbf{M} = \begin{bmatrix} \mathbf{P} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{Q} \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} \mathbf{H}_{11} & \mathbf{H}_{12} \\ \mathbf{H}_{21} & \mathbf{H}_{22} \end{bmatrix}, \quad (7)$$

where the blocks' elements are defined as follows:

- $P_{kj}^e = \int_e \phi_j^e \phi_k^e ds;$
- $R_{kv}^e = \int_e \phi_v \phi_k^e ds;$
- $Q_{v'v} = \sum_{e \in \mathcal{E}} \int_e \phi_v \phi_{v'} ds;$
- $(H_{11})_{kj}^e = \int_e \left[\frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_k^e}{\partial s} + V_e \phi_j^e \phi_k^e \right] ds;$
- $(H_{12})_{vj}^e = \int_e \left[\frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_v}{\partial s} + V_e \phi_j^e \phi_v \right] ds;$
- $(H_{22})_{v'v} = \sum_{e \in \mathcal{E}} \int_e \left[\frac{\partial \phi_v}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + V_e \phi_v \phi_{v'} \right] ds.$

Theorem 2.1. *The mass matrix $\mathbf{M}$ and the discrete Hamiltonian $\mathbf{H}$ have the following block structures:*

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{11} & \mathbf{H}_{12} \\ \mathbf{H}_{12}^\top & \mathbf{H}_{22} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{G} \end{bmatrix}}_{\mathbf{L}} + \underbrace{\begin{bmatrix} \mathbf{V} & \mathbf{C} \\ \mathbf{C}^\top & \mathbf{D} \end{bmatrix}}_{\mathbf{M}_v},$$

$$\mathbf{M} = \begin{bmatrix} \mathbf{P} & \mathbf{R} \\ \mathbf{R}^\top & \mathbf{Q} \end{bmatrix},$$

where the entries of each block are defined as follows.

In particular:

1. $\mathbf{H}$ is symmetric and positive definite; $\mathbf{M}$ is symmetric positive definite.
2. $\mathbf{A}$ and $\mathbf{V}$ are block diagonal matrices, with one block per edge $e \in \mathcal{E}$, of size $(n_e - 1) \times (n_e - 1)$. Similarly, $\mathbf{P}$ is block diagonal with one block per edge.
3. $\mathbf{G}$ and $\mathbf{D}$ are diagonal matrices of size $N \times N$, being $N = |\mathcal{V}|$. Similarly, $\mathbf{Q}$ is diagonal.
4. $\mathbf{L}$, called the *stiffness matrix*, is the Laplace matrix of the extended graph. $\mathbf{M}_V$, called the *generalized mass matrix*, encodes the potential V . The full discrete Hamiltonian satisfies $\mathbf{H} = \mathbf{L} + \mathbf{M}_V$.
5. The entries of each block of $\mathbf{H}$ are:

$$A_{jk} = \int_e \frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_k^e}{\partial s} ds = \begin{cases} 2/h_e & \text{if } j = k, \\ -1/h_e & \text{if } |j - k| = 1, \\ 0 & \text{otherwise,} \end{cases}$$

$$B_{vk} = \int_e \frac{\partial \phi_v}{\partial s} \frac{\partial \phi_k^e}{\partial s} ds = \begin{cases} -1/h_e & \text{if } e \in \mathcal{E}_v \cap \mathcal{E}_k \neq \emptyset, \\ 0 & \text{otherwise,} \end{cases}$$

$$G_{vv} = \int_{\mathcal{W}_v} \frac{\partial \phi_v}{\partial s} \frac{\partial \phi_v}{\partial s} ds = \sum_{e \in \mathcal{E}_v} \frac{1}{h_e},$$

$$V_{jk} = \int_e V(s) \phi_j^e(s) \phi_k^e(s) ds, \quad C_{vk} = \int_e V(s) \phi_v(s) \phi_k^e(s) ds, \quad D_{vv} = \int_{\mathcal{W}_v} V(s) \phi_v^2(s) ds.$$

6. The entries of each block of $\mathbf{M}$ are:

$$P_{jk}^e = \int_e \phi_j^e \phi_k^e ds = \begin{cases} 2h_e/3 & \text{if } j = k, \\ h_e/6 & \text{if } |j - k| = 1, \\ 0 & \text{otherwise,} \end{cases}$$

$$R_{kv}^e = \int_e \phi_v \phi_k^e ds = \begin{cases} h_e/6 & \text{if } e \in \mathcal{E}_v \text{ and } k \text{ is the interior node adjacent to } v, \\ 0 & \text{otherwise,} \end{cases}$$

$$Q_{v'v} = \sum_{e \in \mathcal{E}} \int_e \phi_v \phi_{v'} ds = \begin{cases} \sum_{e \in \mathcal{E}_v} \frac{h_e}{3} & \text{if } v' = v, \\ 0 & \text{otherwise.} \end{cases}$$

The diagonal structure of $\mathbf{Q}$ follows from the fact that vertex basis functions ϕ_v and $\phi_{v'}$ have disjoint support whenever $v \neq v'$, since each ϕ_v is supported only on edges incident to v .

2.3 Fully discrete system

Discretizing the semi-discrete system in time by the θ -method with uniform step $\Delta t = T/N_T$ and setting $\mathbf{u}^n \approx \mathbf{u}(t^n)$, one obtains

$$\mathbf{M} \frac{\mathbf{u}^n - \mathbf{u}^{n-1}}{\Delta t} + \mathbf{H}(\theta \mathbf{u}^n + (1 - \theta) \mathbf{u}^{n-1}) = \theta \mathbf{f}^n + (1 - \theta) \mathbf{f}^{n-1},$$

which rearranges to

$$\mathbf{M}_\theta \mathbf{u}^n = (\mathbf{M} - (1 - \theta) \Delta t \mathbf{H}) \mathbf{u}^{n-1} + \Delta t (\theta \mathbf{f}^n + (1 - \theta) \mathbf{f}^{n-1}), \quad (8)$$

where

$$\mathbf{M}_\theta = \mathbf{M} + \theta \Delta t \mathbf{H} = \begin{bmatrix} \mathbf{P} + \theta \Delta t \mathbf{H}_{11} & \mathbf{R} + \theta \Delta t \mathbf{H}_{12} \\ \mathbf{R}^\top + \theta \Delta t \mathbf{H}_{21} & \mathbf{Q} + \theta \Delta t \mathbf{H}_{22} \end{bmatrix}.$$

The choice $\theta = 1$ recovers the backward Euler method, while $\theta = \frac{1}{2}$ gives the Crank–Nicolson scheme.

Since $a(\cdot, \cdot)$ is symmetric and coercive on $H^1(\Gamma)$ and $\mathbf{M}$ is symmetric positive definite, $\mathbf{M}_\theta$ is symmetric positive definite for every $\Delta t > 0$ and $\theta \in (0, 1]$; hence the linear system admits a unique solution at each time step.

3 Numerical Benchmarks

This section presents the numerical benchmarks developed to validate the C++ implementation described in Chapter 7. The tests are organized as a progressive validation path, moving from basic elliptic checks to genuinely parabolic phenomena. Following [5], we begin with three elliptic test cases, denoted E1–E3, which serve as reference benchmarks for the parabolic extension. Test E1 verifies the correctness of the assembly procedure and the basic control flow by prescribing a constant exact solution. Test E2 assesses the enforcement of mixed Dirichlet–Neumann–Kirchhoff boundary conditions through a linear exact solution. Test E3 investigates the spatial convergence of the solver by adopting an exact solution that does not belong to the finite element space (a sinusoidal solution). We then introduce three parabolic benchmarks. Test 1 considers a radially decaying solution on a star-shaped graph, where both diffusion and the reaction term contribute to the time evolution of the solution. Test 2 assesses temporal convergence, verifying that the Backward Euler scheme achieves first-order accuracy and the Crank–Nicolson scheme achieves second-order accuracy in time. Test 3 prescribes a spatially localized initial condition and examines the qualitative propagation of the solution across the graph topology.

3.1 Elliptic Test Cases

We begin with three elliptic benchmark problems on a star-shaped metric graph Γ with five vertices and four edges. In particular, $\Gamma = (\mathcal{V}, \mathcal{E})$, with $\mathcal{V} = \{v_0 = (0, 0), v_1 = (1, 0), v_2 = (0, 1), v_3 = (-1, 0), v_4 = (0, -1)\}$ and $\mathcal{E} = \{\{v_0, v_1\}, \{v_0, v_2\}, \{v_0, v_3\}, \{v_0, v_4\}\}$. Thus, Γ consists of four unit-length edges meeting at the central vertex v_0 , like the one represented in Section 1.2.

A key observation is the following: these benchmarks, while representing elliptic cases, are used to validate a parabolic pipeline, so they will be interpreted in this setting.

3.1.1 Test E1: constant function

The first elliptic test considers a constant exact solution on the star graph:

$$u_{\text{exact}} = 5,$$

satisfying the following:

$$\frac{\partial u}{\partial t} - \nabla u = 0.$$

Boundary data and the initial condition are chosen consistently so that $u_{\text{exact}} = 5$ on every edge of Γ . This test checks the basic assembly procedure and control flow of the code for a trivial case. At final time $T = 1$, the numerical solution remains constant on the whole graph, with value $u = 5$ up to roundoff errors. In this simple case, the numerical algorithms are expected to return the exact solution. This is indeed the case, as shown in the following Figure. A mesh with 100 elements per edge is used to discretize the graph. The graph is discretized again using 100 elements per edge. The final time of observation is $T = 1$, the time step is $\Delta t = 0.05$ and the Backward Euler scheme is used.

Solution at t = 1.00

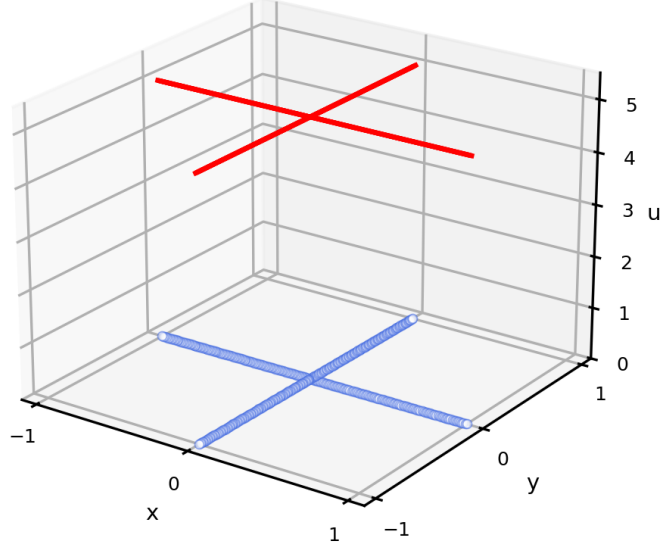


Figure 1: Test E1. Constant solution at final time $T = 1$, with the computational star graph drawn in blue at height zero and the numerical solution lifted in red. The flat profile confirms that $u = 5$ is preserved on every edge of the graph.

3.1.2 Test E2: linear function

Let us consider the parabolic problem

$$\begin{cases} \frac{\partial u}{\partial t} - \Delta u = 0 & \text{in } \Gamma \times (0, T), \\ \sum_{e \in \mathcal{E}_{v_0}} \frac{\partial u_e}{\partial n_e}(v_0) = 0, \\ u(v_1, t) = 1, & t \in (0, T), \\ u(v_2, t) = 1, & t \in (0, T), \\ u(v_3, t) = -1, & t \in (0, T), \\ u(v_4, t) = -1, & t \in (0, T), \\ u(x, y, 0) = x + y, & \text{on } \Gamma. \end{cases}$$

The exact solution is stationary and is given by

$$u_{\text{ex}}(x, y) = x + y.$$

Since the exact solution is linear on each edge, it belongs to the finite element space of continuous piecewise linear functions. Therefore, the numerical method is expected to reproduce it up to roundoff errors. The graph is discretized again using 100 elements per edge. The final time is still $T = 1$, the time step is $\Delta t = 0.05$ and the Backward Euler scheme is used. This benchmark proves that our code can handle various boundary conditions.

Solution at t = 1.00

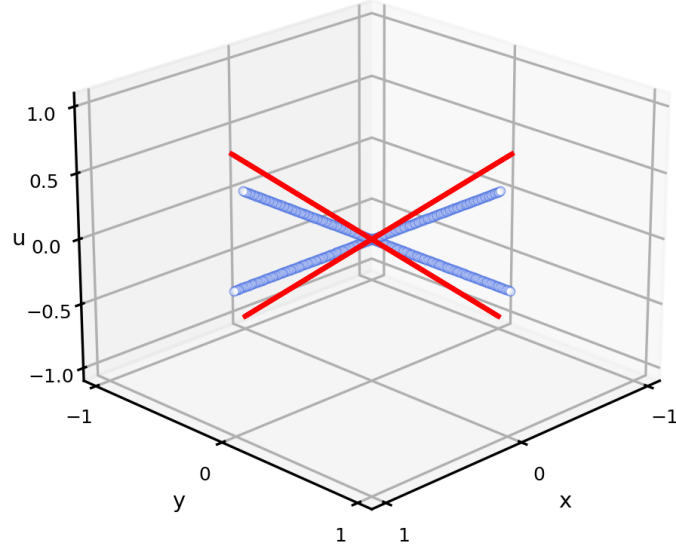


Figure 2: Test E2. Linear solution at final time $T = 1$. The stationary profile $u = x + y$ is preserved on the whole graph: the solution is linear with unit slope on all four branches, running from -1 to 1 along both the horizontal and the vertical path and continuous at the central vertex.

3.1.3 Test E3: function of a single variable

The third elliptic test considers a non-polynomial function depending only on the horizontal coordinate:

$$u_{\text{ex}}(x, y) = \sin(2\pi x).$$

This solution is not exactly represented by the piecewise linear finite element space. Therefore, unlike the previous two tests, the numerical error is nonzero and should decrease as the mesh is refined. This test is useful to observe the spatial convergence properties of the method.

On the star-shaped graph Γ , we solve

$$\begin{cases} \partial_t u - \Delta u = 4\pi^2 \sin(2\pi x) & \text{in } \Gamma \times (0, T), \\ u(v, t) = 0 & \forall v \in \mathcal{V}, t \in (0, T), \\ u(x, y, 0) = \sin(2\pi x) & \text{on } \Gamma. \end{cases}$$

Unlike the constant and linear benchmarks, the exact solution is not contained in the finite element space of continuous piecewise linear functions. Therefore the numerical error is not expected to vanish, but it should decrease as the mesh is refined. The final time is $T = 1$, the time step is $\Delta t = 0.005$ and the Backward Euler method is used.

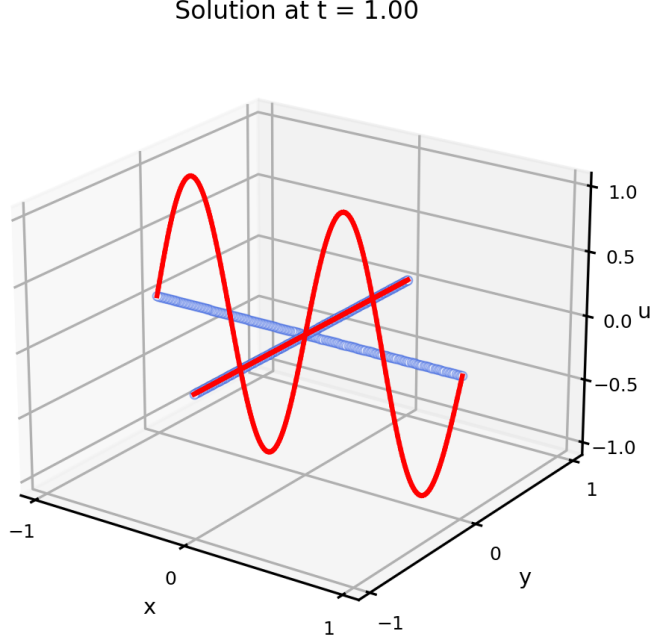


Figure 3: Test E3. Numerical solution associated with $u_{\text{ex}}(x, y, t) = \sin(2\pi x)$, lifted in red above the star graph. The two full periods develop along the horizontal branch, the solution is zero on the vertical branches, where $x = 0$, and the discrete solution is correctly globally continuous at the central vertex, where the sine vanishes.

To study the spatial convergence, the problem is solved using 10, 20, 40, and 80 elements per edge. The time step is chosen as $\Delta t = 0.025h^2$, so that the temporal error is negligible with respect to the spatial error. Since continuous piecewise linear finite elements are used, the interpolation error for a sufficiently regular exact solution satisfies the classical estimate: $\|u - u_h\|_{L^2(\Gamma)} \leq Ch^2\|u\|_{H^2(\Gamma)}$. Therefore, when the temporal error is made negligible by choosing $\Delta t = 0.025h^2$, the dominant contribution to the error is the spatial finite element error. Hence, second-order convergence in the L^2 -norm is expected. The numerical results confirm this behaviour, since the observed convergence rates are approximately equal to 2.

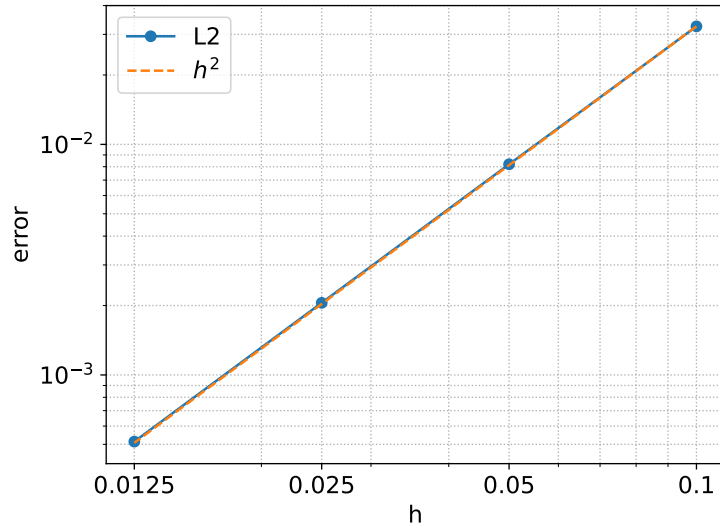


Figure 4: Test E3. Spatial convergence for the parabolic sine benchmark. The results show second-order convergence in the L^2 -norm.

3.2 Parabolic Test Cases

We now present the parabolic benchmarks implemented in this project.

3.2.1 Test 1: radially decaying solution

This benchmark introduces a genuinely time-dependent exact solution. We solve

$$\partial_t u - \Delta u + u = 0 \quad \text{on } \Gamma \times (0, T).$$

The exact solution on each edge is

$$u_{\text{ex}}(s, t) = \exp \left[- \left(\frac{\pi^2}{4} + 1 \right) t \right] \cos \left(\frac{\pi s}{2} \right).$$

Homogeneous Dirichlet conditions are imposed at the four external vertices, while the central vertex satisfies the natural Kirchhoff condition. This benchmark validates the time-dependent part of the solver.

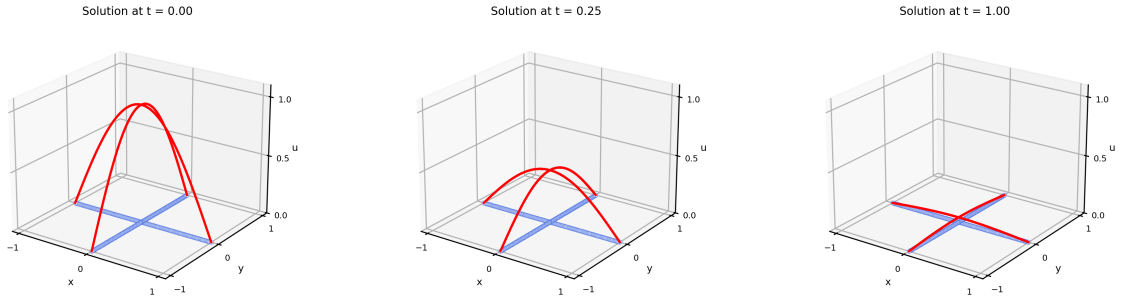


Figure 5: This test considers a radially decaying solution where both the diffusion and reaction term contribute to the temporal dynamics of the solution: the initial bump $\cos(\pi s/2)$, equal to one at the central vertex and pinned at zero at the four external vertices, decays uniformly on the star. The Crank–Nicolson scheme ($\theta = 1/2$) is used, with $\Delta t = 0.05$ and a mesh size $h = 0.01$.

For the temporal convergence test, a fine spatial mesh with 320 elements per edge is used. Since each edge has unit length, the mesh size is $h = \frac{1}{320} = 0.003125$. This choice makes the spatial error negligible with respect to the temporal error. Indeed, for piecewise linear finite elements the L^2 -error behaves as $O(h^2)$, and $h^2 \simeq 9.77 \cdot 10^{-6}$. On the other hand, the smallest time step used in the test is $\Delta t = 0.025$, for which $\Delta t^2 = 6.25 \cdot 10^{-4}$. Therefore, even for the second-order Crank–Nicolson method, the temporal error is expected to dominate the spatial one.

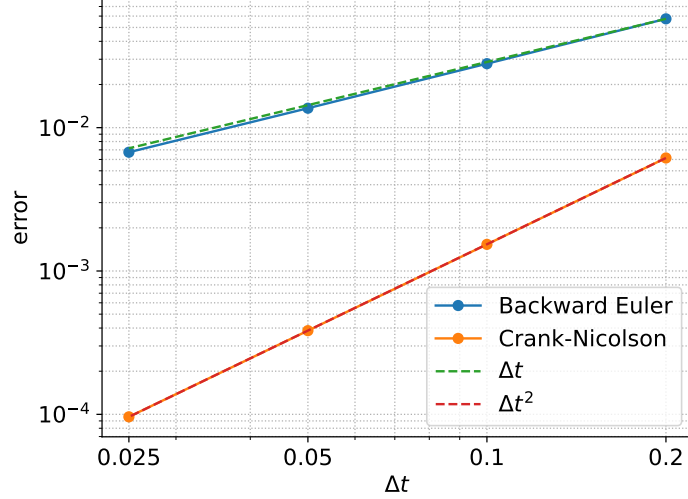


Figure 6: Test 4. Temporal convergence of Backward Euler and Crank–Nicolson. The results show first-order convergence for Backward Euler and second-order convergence for Crank–Nicolson.

3.3 Elliptic eigenvalue problems

The following numerical experiments consider the eigenvalue problem for the Laplacian operator on metric graphs, endowed with Neumann–Kirchhoff conditions at the vertices. Three different graph topologies are analyzed in order to compare our numerical results with the reference ones reported in [6]. The comparison is understood up to the sign and normalization of the eigenfunctions. Moreover, in the case of multiple eigenvalues, the computed eigenfunctions may differ by linear combinations within the same eigenspace.

3.3.1 Test 1: four-pointed star graph

The goal of this benchmark, as for the following ones, is to verify that the code correctly solves generalized eigenvalue problems on metric graphs. This also represents the first step of the parabolic pipeline, since an eigenfunction of the graph Laplacian provides an exact modal solution of the heat equation. The first topology considered is the four-pointed star graph. We compute the eigenpairs (λ, u) satisfying

$$-\Delta u = \lambda u \quad \text{on } \Gamma,$$

with homogeneous Neumann conditions at the four external vertices and the Neumann–Kirchhoff condition at the central vertex. Each edge has length 1 and is discretized using 100 finite elements. Fig. 7 shows the first six eigenmodes. The blue lines represent the metric graph, while the red curves represent the corresponding eigenfunctions lifted in the vertical direction. The eigenfunctions are rescaled only for visualization purposes; the eigenvalues are those computed by the finite element generalized eigenvalue problem. The results are in good agreement with the reference results reported on page 31 of [6].

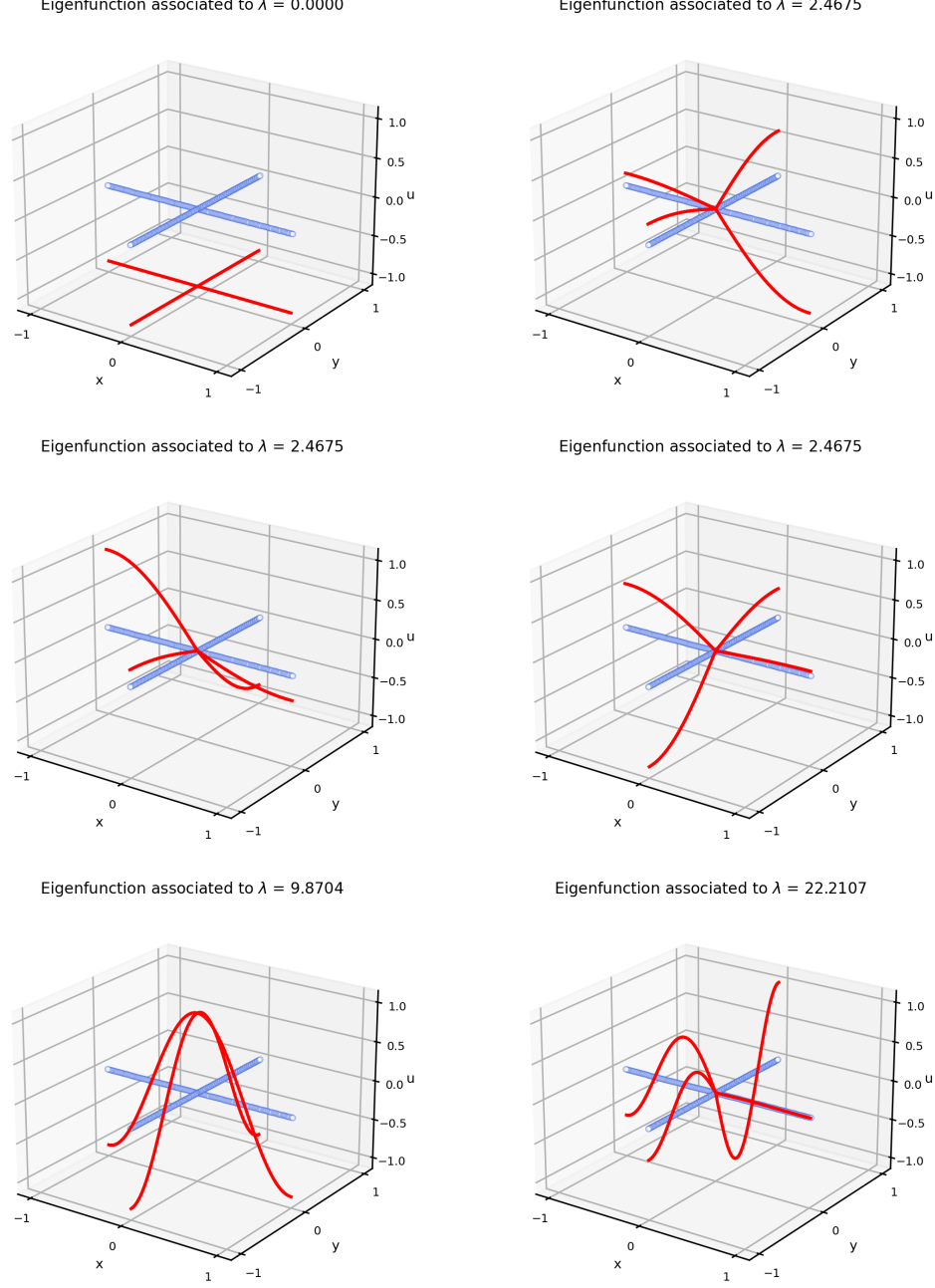


Figure 7: First six eigenmodes of the Laplacian on the four-pointed star graph. The blue graph represents the computational domain, while the red curves show the corresponding eigenfunctions. The vertical scaling is used only for visualization.

The eigenvectors computed by the finite-element solver are normalized with respect to the mass matrix, namely $\mathbf{u}^T M \mathbf{u} = 1$, which corresponds to the discrete $L^2(\Gamma)$ -normalization. In the figures, however, each eigenfunction is additionally rescaled by its maximum absolute nodal value only for visualization purposes.

3.3.2 Test 2: graphene-like graph

The second benchmark considers a graphene-like metric graph. This topology is more complex than the four-pointed star graph, since it contains several internal junctions and cycles. The aim is to test our solver on a graph with a richer connectivity structure, while keeping the same differential operator and vertex conditions introduced in the previous test. We compute the eigenpairs (λ, u)

satisfying

$$-\Delta u = \lambda u \quad \text{on } \Gamma,$$

with homogeneous Neumann conditions at the external vertices and Neumann–Kirchhoff conditions at the internal vertices. The graph has 12 vertices and 13 edges. Each edge has length 1 and is discretized using 100 elements 100 per edge.

Fig. 8 shows the first six eigenmodes, while Figure 9 shows the following four. The blue lines still represent the metric graph, whereas the red curves represent the computed eigenfunctions lifted in the vertical direction. As in the previous test, the vertical scaling is used only for visualization purposes. The two related images show good agreement with [6].

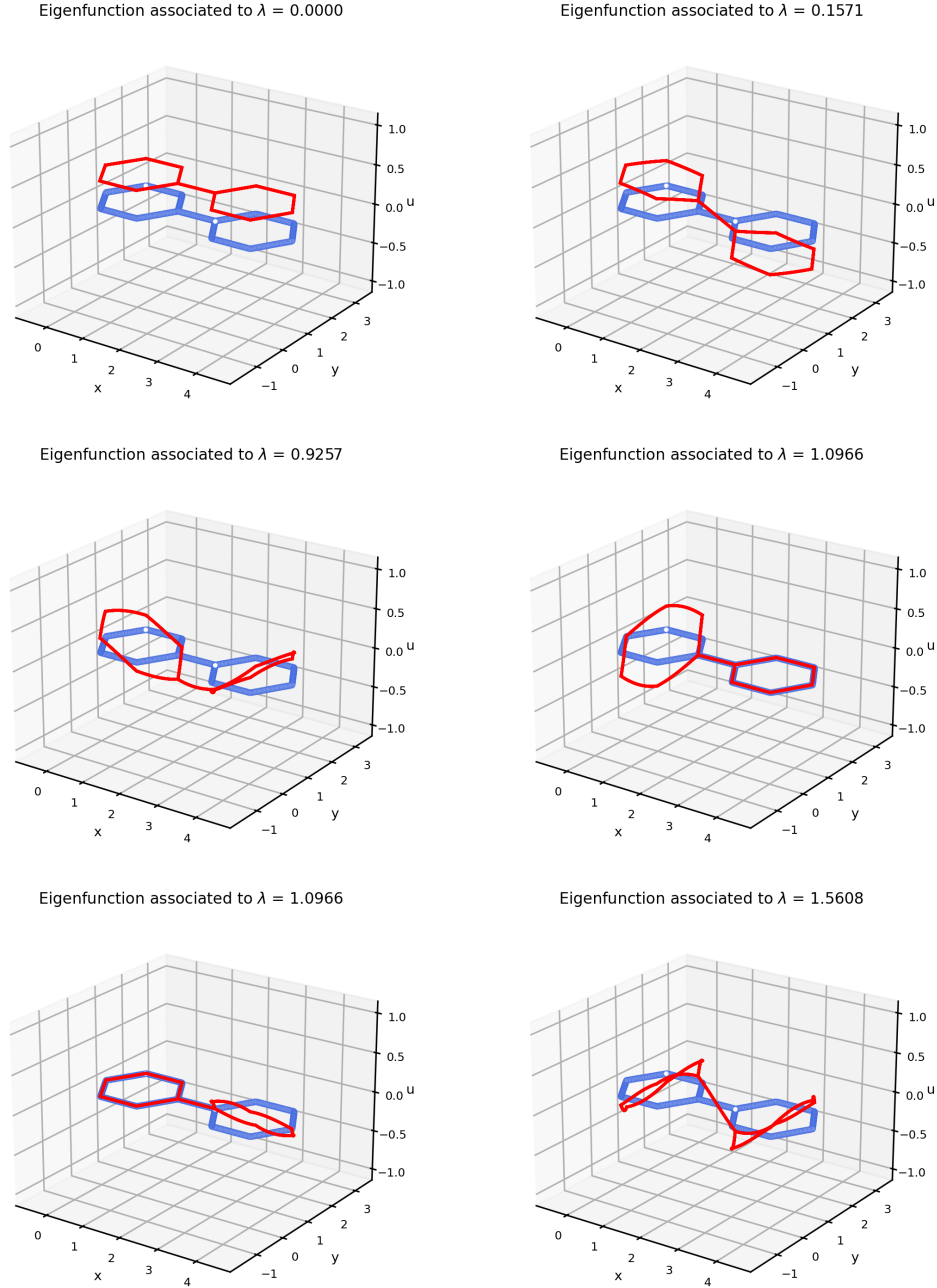


Figure 8

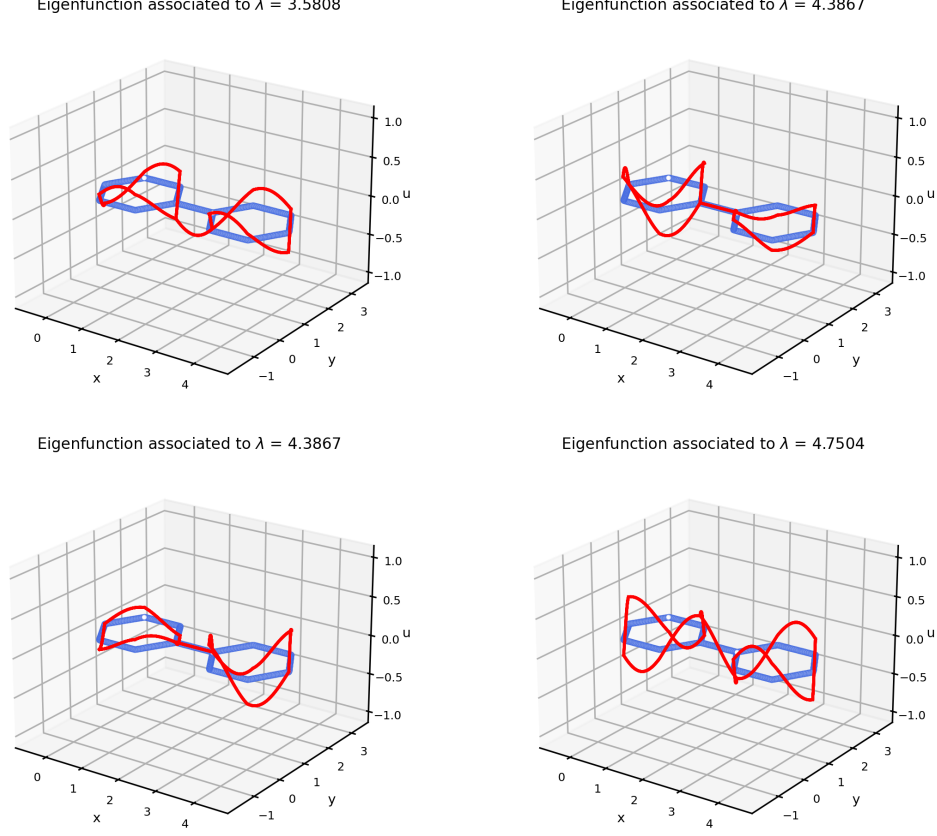


Figure 9: Fig. 8 and 9 show the first ten eigencouples of the Laplacian operator on a graphene-like graph. In blue, the domain of the problem. Results are scaled.

3.3.3 Test 3: binary tree graph

The third benchmark considers a binary tree metric graph. This topology is particularly relevant because it contains several branching points and many terminal vertices, while remaining simpler than a general network. It is therefore a useful intermediate test between elementary graph topologies and more complex network-like domains.

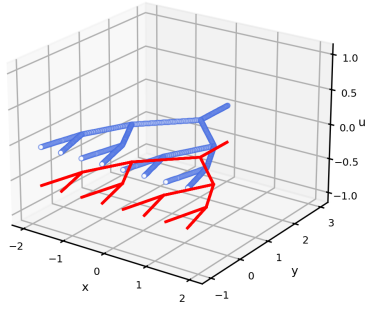
We compute again the eigenpairs (λ, u) satisfying

$$-\Delta_{\Gamma} u = \lambda u \quad \text{on } \Gamma,$$

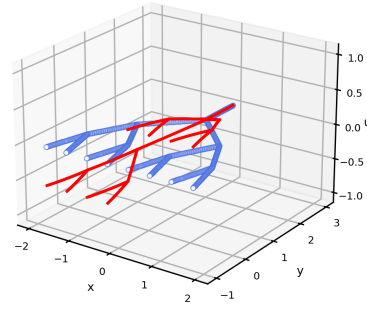
with homogeneous Neumann conditions at the leaves and Neumann–Kirchhoff conditions at the internal ones. The graph has 16 vertices and 15 edges. Each edge has length 1 and is discretized using 100 finite elements as in the previous cases.

Fig. 10 and Fig. 11 show the first sixteen eigenmodes. As before, the blue lines represent the metric graph and the red curves represent the eigenfunctions lifted in the vertical direction. The vertical scaling is used only for visualization. This experiment shows a very good agreement with the reference spectral behaviour reported in [6].

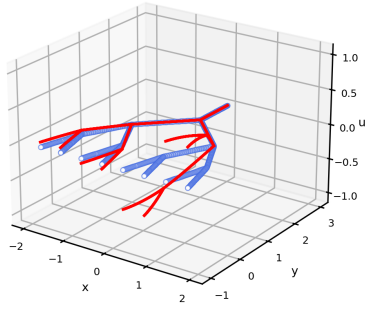
Eigenfunction associated to $\lambda = 0.0000$



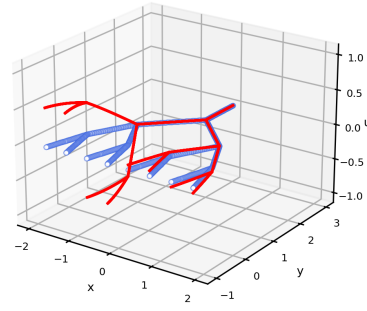
Eigenfunction associated to $\lambda = 0.1155$



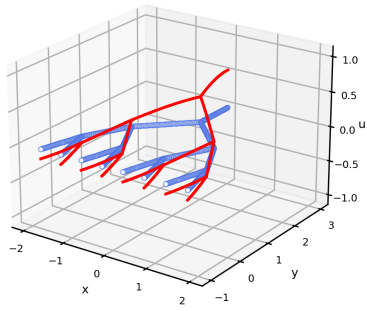
Eigenfunction associated to $\lambda = 0.3788$



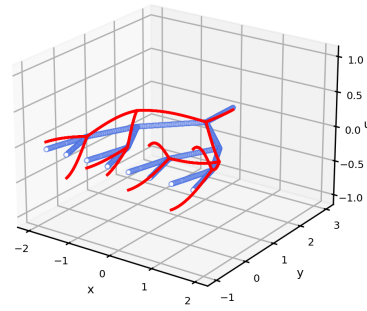
Eigenfunction associated to $\lambda = 0.3788$



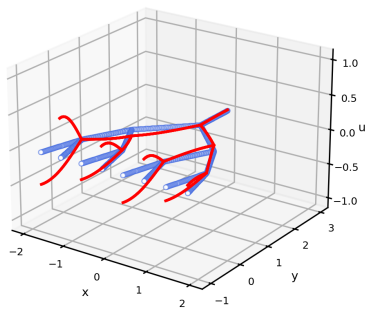
Eigenfunction associated to $\lambda = 0.7074$



Eigenfunction associated to $\lambda = 2.4675$



Eigenfunction associated to $\lambda = 2.4675$



Eigenfunction associated to $\lambda = 2.4675$

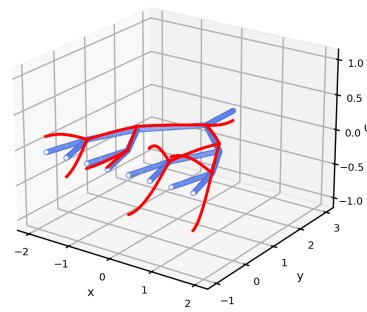


Figure 10: First eight eigenmodes of the Laplacian on the binary tree graph.

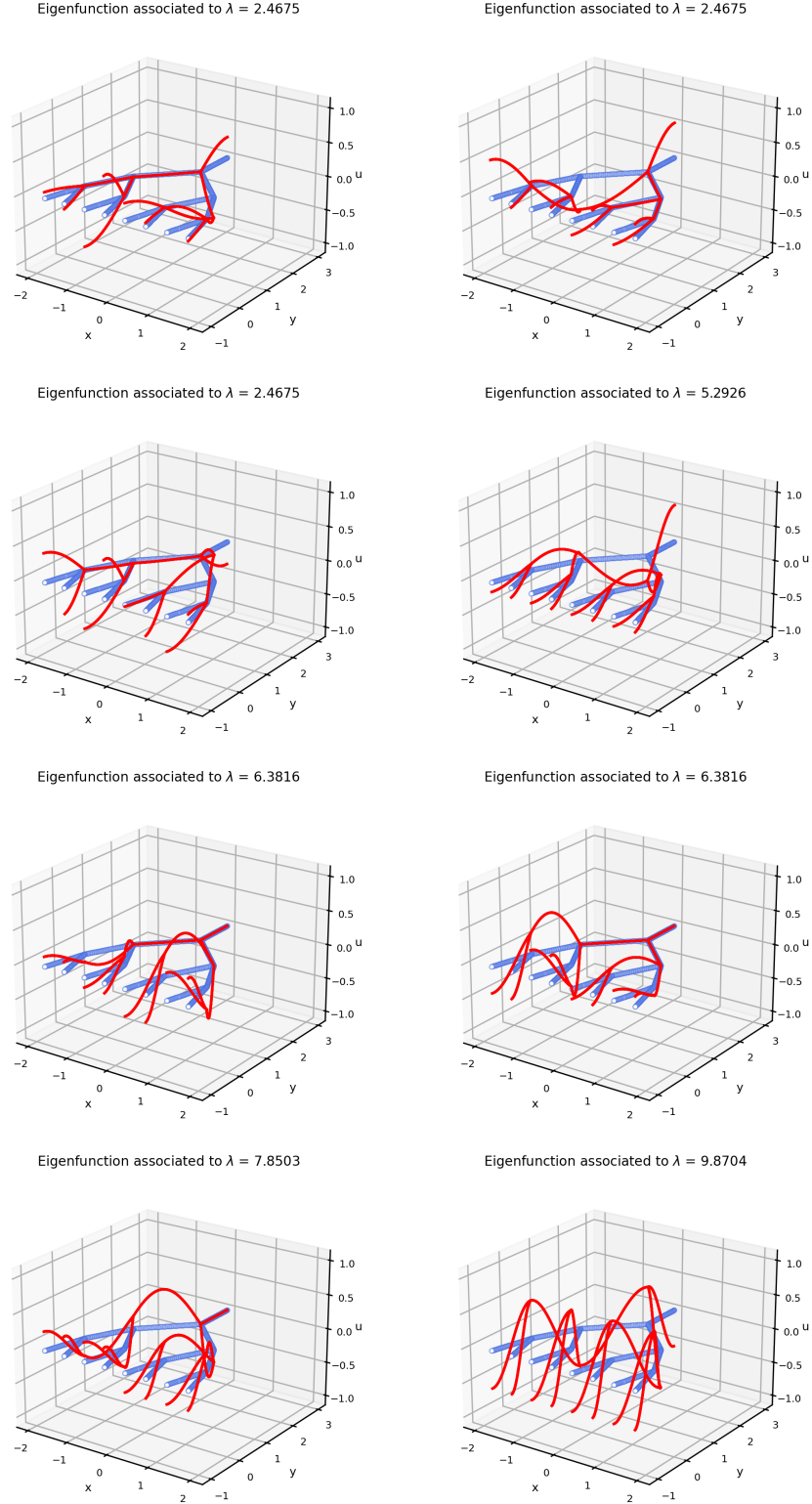


Figure 11: Eigenmodes 8–15 of the Laplacian on the binary tree graph.

Remark. In all the experiments, eigenfunctions associated with larger eigenvalues display a progressively more oscillatory behaviour along the graphs. This is consistent with the spectral properties of the Laplacian operator: the operator is unbounded, meaning that its spectrum is not bounded from above and $\lambda_k \rightarrow +\infty$ as $k \rightarrow \infty$; therefore, higher eigenvalues correspond to higher-frequency modes on the metric graph.

3.4 Parabolic eigenvalue problems

The eigenvalue computations of the previous section can be used to construct exact solutions for the heat equation on metric graphs. Let u_k be an eigenmode of the graph Laplacian associated with the eigenvalue λ_k , namely

$$-\Delta u_k = \lambda_k u_k \quad \text{on } \Gamma,$$

with Neumann–Kirchhoff vertex conditions. Then the heat equation

$$\partial_t u - \mu \Delta u = 0 \quad \text{on } \Gamma \times (0, T]$$

admits the exact modal solution

$$u(x, t) = e^{-\mu \lambda_k t} u_k(x).$$

This provides a useful validation test for the parabolic solver, since both the spatial structure and the temporal decay are known explicitly. The initial datum is a normalized eigenmode, so that

$$\|u_k\|_{L^2(\Gamma)} = 1.$$

Consequently, the exact evolution of the L^2 -norm is

$$\|u(t)\|_{L^2(\Gamma)} = e^{-\mu \lambda_k t}.$$

In the following tests we take $\mu = 1.0$, $T = 1$, $\Delta t = 0.01$, and use the Crank–Nicolson method.

3.4.1 Test 4: heat equation on the graphene-like graph

For the graphene-like graph we select the sixth eigenmode, associated with $\lambda_6 \simeq 3.5808$. Fig. 12 shows the numerical solution at selected time instants. The shape of the selected eigenmode is preserved, while its amplitude decays in time. Fig. 13 compares the numerical L^2 -norm with the exact decay law. Since the initial eigenmode is mass-normalized, the curve starts from 1. At $T = 1.0$, the numerical norm is approximately $2.7842 \cdot 10^{-2}$, in excellent agreement with the exact value $2.7853 \cdot 10^{-2}$.

3.4.2 Test 5: heat equation on the binary tree graph

For the binary tree graph we select the fifth eigenmode, associated with $\lambda_5 \simeq 2.4675$. Fig. 14 shows the time evolution of the numerical solution. Also in this case, the eigenmode profile is preserved and only its amplitude decays. Fig. 15 shows the agreement between the numerical L^2 -norm and the exact exponential decay. At $T = 1.0$, the numerical norm is approximately $8.4790 \cdot 10^{-2}$, while the exact value is $8.4801 \cdot 10^{-2}$.

3.4.3 Laplace matrix spectral comparison

It can be interesting to compare the spectrum of the combinatorial Laplacian of the original graph Γ with the spectrum of the Laplace matrix associated with the corresponding extended graph $\mathcal{G}$. Given a graph with N vertices, the combinatorial Laplacian, introduced in Definition 1.6 (and described in App. A), is

$$L = D - A,$$

where D is the degree matrix and A is the adjacency matrix. In our implementation, this matrix is assembled through a custom C++ routine from the graph connectivity. Since L is real and symmetric, its eigenvalues are then computed using Eigen's `SelfAdjointEigenSolver()`.

For the extended graph, the finite element discretization of the Laplace eigenvalue problem leads to the generalized algebraic problem

$$H\mathbf{u} = \lambda M\mathbf{u},$$

where H is the stiffness matrix and M is the mass matrix. This problem is solved using Eigen's `GeneralizedSelfAdjointEigenSolver()`. This choice is motivated by the fact that H and M are symmetric matrices and M is positive definite.

The comparison is performed on three graph topologies: the four-pointed star graph, the graphene-like graph and a family of tree graphs. The extended graph is constructed using 100 finite elements per edge. This choice makes the spectral comparison consistent with the eigenfunction plots

Heat equation on graphene, sixth eigenmode, $\lambda = 3.5808$

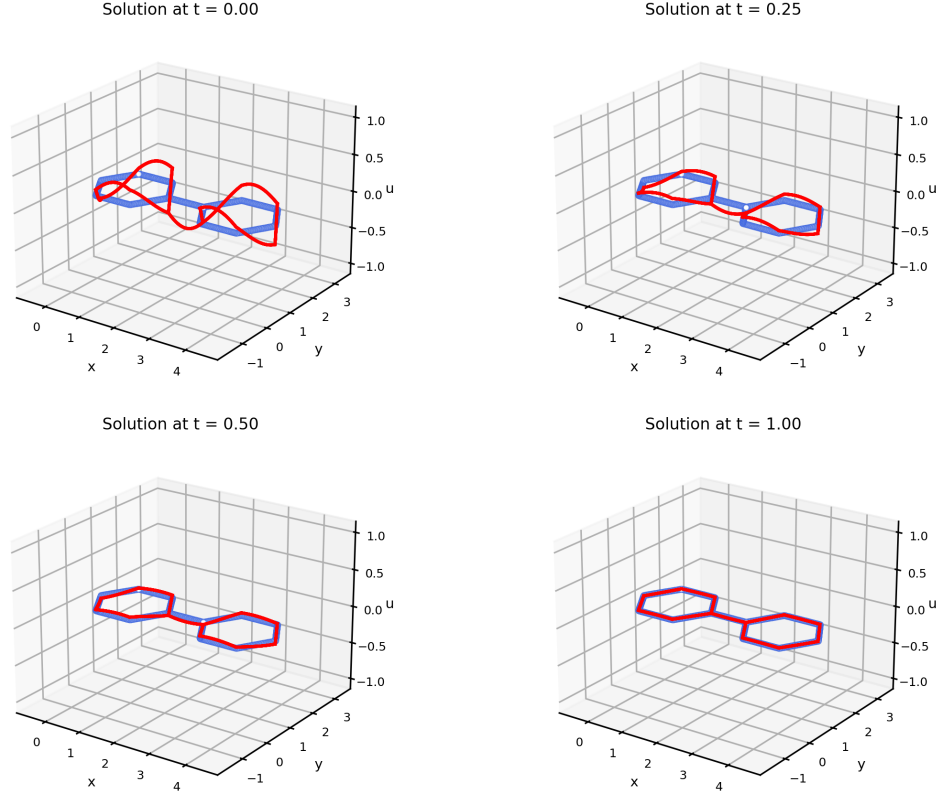


Figure 12: Heat equation on the graphene-like graph with initial datum given by the sixth eigenmode, associated with $\lambda_6 \simeq 3.5808$.

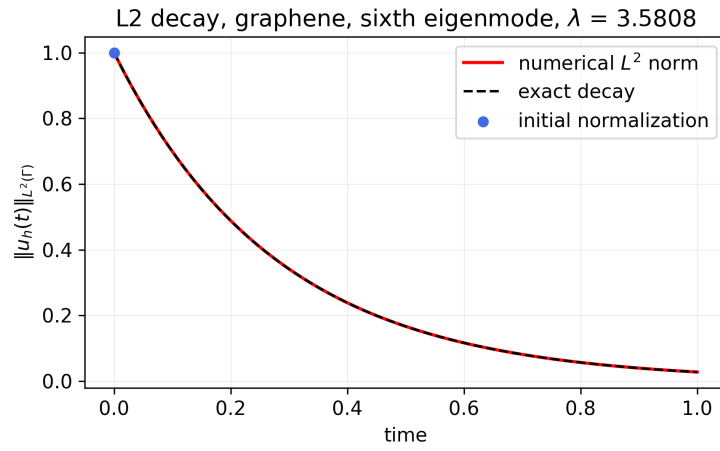


Figure 13: Decay of the numerical L^2 -norm compared with the exact law for the graphene-like graph.

Heat equation on tree, fifth eigenmode, $\lambda = 2.4675$

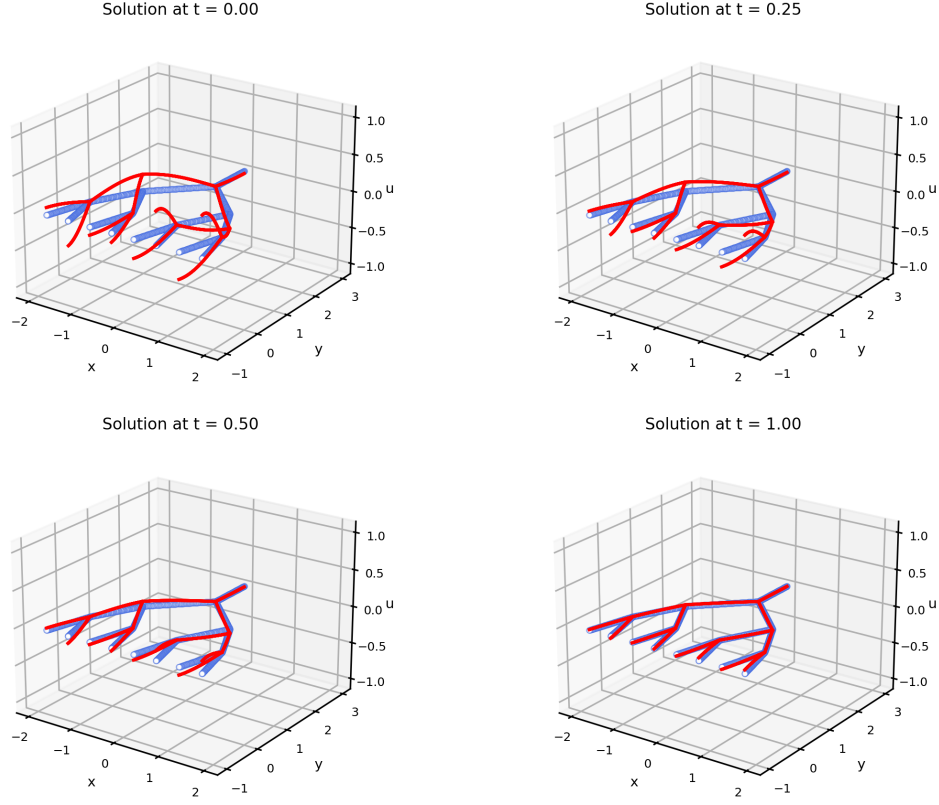


Figure 14: Heat equation on the binary tree graph with initial datum given by the fifth eigenmode, associated with $\lambda_5 \simeq 2.4675$.

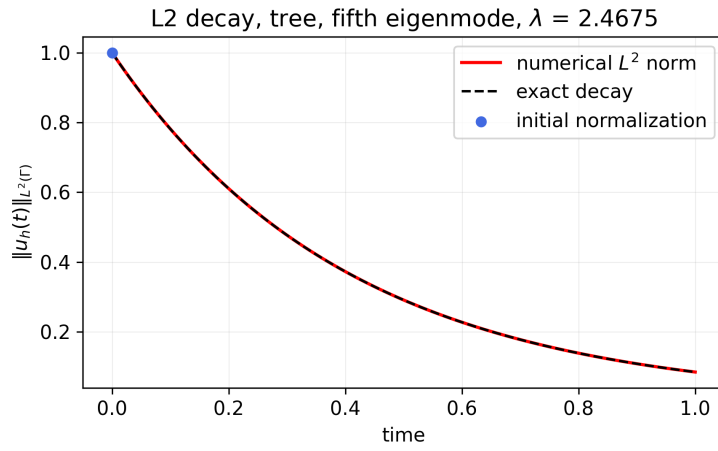


Figure 15: Decay of the numerical L^2 -norm compared with the exact law for the binary tree graph.

presented in the previous benchmarks.

Fig. 16 reports the comparison between the eigenvalues of the combinatorial Laplace matrix and those of the extended Laplace matrix. The corresponding numerical values are given in Tables 1, 2 and 3.

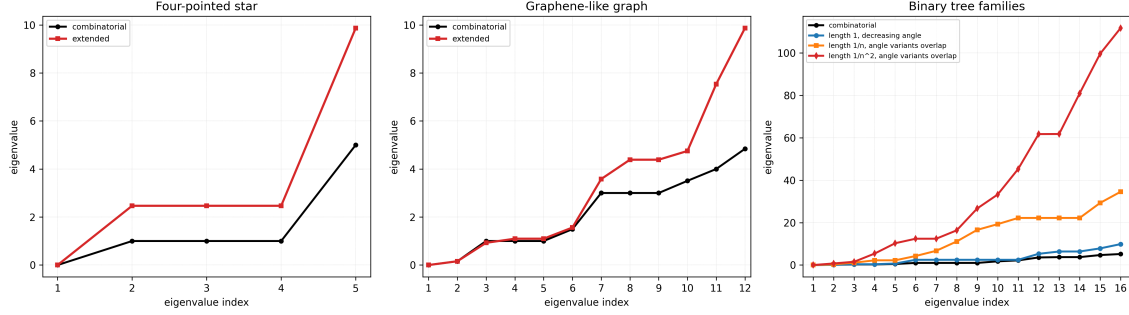


Figure 16: Comparison between the spectrum of the combinatorial Laplace matrix of the original graph and the spectrum of the extended Laplace matrix. Eigenvalues are plotted against their index. The presence of symmetries in the graph topology causes the existence of multiple eigenvalues.

Combinatorial Laplace matrix	Extended Laplace matrix
0	0
1.0000	2.4675
1.0000	2.4675
1.0000	2.4675
5.0000	9.8704

Table 1: Spectral comparison for the four-pointed star graph.

Combinatorial Laplace matrix	Extended Laplace matrix
0	0
0.1578	0.1571
1.0000	0.9257
1.0000	1.0966
1.0000	1.0966
1.4931	1.5608
3.0000	3.5808
3.0000	4.3867
3.0000	4.3867
3.5069	4.7504
4.0000	7.5371
4.8422	9.8704

Table 2: Spectral comparison for the graphene-like graph.

For the tree graph, several geometric configurations are considered. The first case has constant edge length and decreasing bifurcation angle. The other cases have either linearly or quadratically (with the bifurcation level) decreasing edge lengths. More precisely, if n denotes the bifurcation level, the edge lengths are chosen as

$$\ell_n = 1, \quad \ell_n = \frac{1}{n}, \quad \ell_n = \frac{1}{n^2}.$$

The numerical experiments show that, for fixed and decreasing edge-length laws, changing the bifurcation angle does not affect the spectrum of the extended Laplace matrix. This is expected, since the metric graph Laplacian depends on the graph topology and on the edge lengths, while the planar drawing angles only affect the visualization of the graph.

The results show that the spectra of the combinatorial and extended Laplace matrices are generally different in value. Nevertheless, the comparison is useful to observe how the graph topology influences

Comb.	$\ell_n = 1$	$\alpha = \pi/4, \ell_n = 1/n$	$\alpha = \pi/4, \ell_n = 1/n^2$
0	0	0	0
0.0968	0.1155	0.3303	0.7426
0.2679	0.3788	1.1186	1.5485
0.2679	0.3788	2.2205	5.4138
0.4965	0.7074	2.2205	10.2825
1.0000	2.4675	4.2573	12.4108
1.0000	2.4675	6.7012	12.4108
1.0000	2.4675	11.1094	16.3523
1.0000	2.4675	16.5789	26.6628
1.7356	2.4675	19.2819	33.2463
2.1939	2.4675	22.2071	45.2803
3.5767	5.2926	22.2071	61.7503
3.7321	6.3816	22.2071	61.7907
3.7321	6.3816	22.2071	80.9487
4.7093	7.8503	29.2923	99.6367
5.1912	9.8704	34.5652	111.7665

Table 3: Spectral comparison for the tree graph. The omitted angle variants give the same extended spectra for fixed edge-length laws.

the low-frequency part of the spectrum. In the tree cases, the dominant effect is produced by the edge lengths: decreasing edge lengths lead to larger eigenvalues, while changing the bifurcation angles does not modify the spectrum.

An additional observation concerns the effect of decreasing edge lengths. When the tree contains shorter and shorter edges, the eigenvalues of the extended Laplace matrix increase significantly. This is consistent with the metric nature of the operator: small edges allow eigenfunctions with rapid spatial oscillations and these high-frequency modes correspond to larger eigenvalues. Therefore, the growth of the spectrum reflects the increasing presence of small geometric scales in the graph.

4 Spreading of neurodegenerative diseases within the brain's connectome

We have validated our finite element library for the solution of graph-based differential problems on a sequence of elliptic and parabolic benchmarks. We now consider a more application-oriented test case. In particular, we study the Fisher–Kolmogorov equation on a brain connectome's topology. This problem provides a natural extension of the previous linear parabolic models, as it combines diffusion along the graph with a nonlinear reaction term. At the same time, it provides a meaningful test for assessing the scalability of our framework through graph reordering and parallel programming techniques, with the particularly interesting perspective of modeling the propagation of infectious misfolded proteins through the brain, a process that is responsible for neurodegenerative diseases. The goal of this section is therefore not to provide a detailed biological study, but to investigate how the developed solver can be extended from controlled benchmark problems to a realistic network-based diffusion-reaction model.

4.1 One-dimensional sensitivity analysis

Prion diseases are characterized by a chain reaction in which misfolded proteins force native ones into a similar pathogenic structure. A physics-based reaction-diffusion model can explain the spreading and growth of these proteins in neurodegenerative diseases: the Fisher-Kolmogorov equation combined with an anisotropic diffusion model [7, 8, 9]. Following the work described in [7], we demonstrate the role of the individual model parameters through a one-dimensional sensitivity test, that we exploit also to validate the nonlinear solver before it touches a connectome. We solve

$$\frac{\partial c}{\partial t} - d \frac{\partial^2 c}{\partial x^2} - \alpha c(1 - c) = 0, \quad (x, t) \in (-1, 1) \times (0, 20], \quad (9)$$

with homogeneous Neumann boundary conditions,

$$\frac{\partial c}{\partial x}(-1, t) = \frac{\partial c}{\partial x}(1, t) = 0.$$

The initial concentration is zero at every mesh node except at the central one, where $c(0, 0) = 0.1$. The interval is discretized using 200 linear finite elements. Backward Euler is employed with $\Delta t = 0.1$ and the nonlinear problem arising at each time step is solved by Newton's method; this is the fully implicit scheme. The sensitivity analysis combines the diffusion coefficients $d \in \{1.0, 2.0, 4.0\} \cdot 10^{-4}$ with the reaction coefficients $\alpha \in \{1.0, 2.0, 4.0\}$. These are the choices made in [7], including the number of elements and the number of time steps. Figure 17 presents the results using an activation-time representation: for every pair (x, c) , the color denotes the first time at which the concentration at position x reaches a prescribed level of c . The black curves are therefore contours of equal activation time. Increasing α accelerates the local conversion of healthy proteins, whereas increasing d enhances their spatial propagation; consequently, both parameters increase the propagation speed, although through different mechanisms. The results are in agreement with the ones of [7].

From this sensitivity test, we can derive the main properties of prion-like diseases:

- the progression of the disease is inevitable if there is an initial infectious seeding;
- the disease progression is characterized by a local increase at the initial seeding region and a subsequent global spread across the domain;
- the critical amount of misfolded proteins (quantity that defines the end of the incubation period) is reached at the seeding site as soon as the initial concentration and growth rate α are higher.

The reference reports a preliminary study in which the time step was varied from $\Delta t/4$ to $4\Delta t$, using $\Delta t = 0.1$, with $\alpha = 2.0$ and $d = 2 \cdot 10^{-4}$. The authors concluded that $\Delta t = 0.1$ was sufficiently small to obtain a converged solution, whereas larger time steps produced a spurious increase in spreading. That study is described in words only: neither the data nor a figure is given, so the used criterion behind the statement cannot be reconstructed. What we can do is to repeat the experiment in

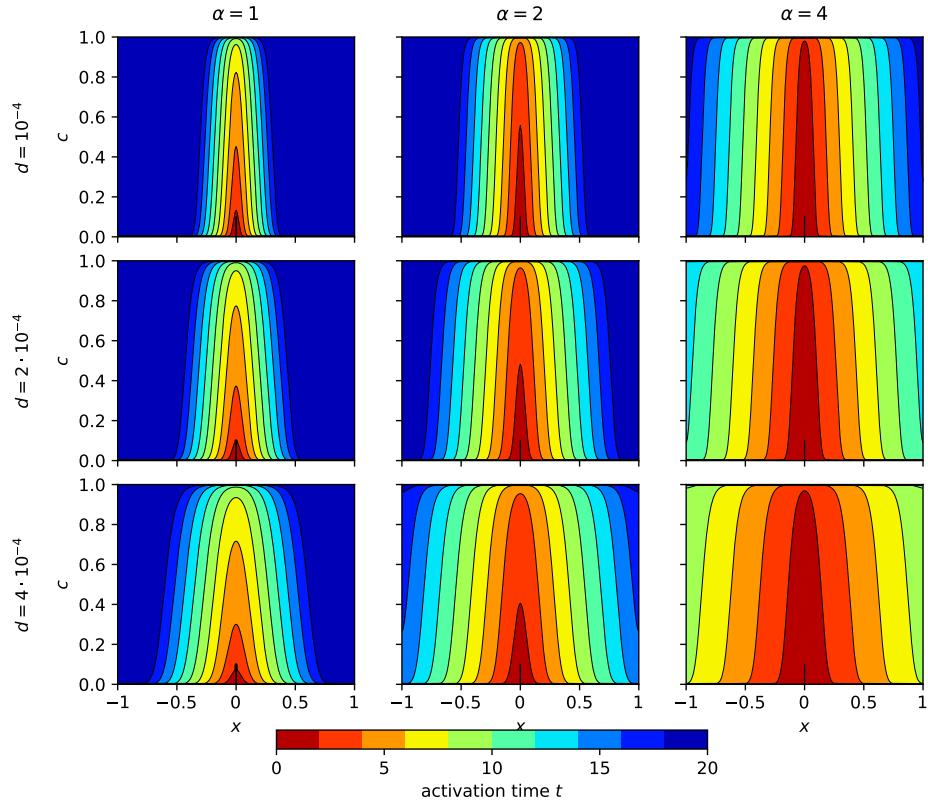


Figure 17: Activation-time maps for the one dimensional Fisher–Kolmogorov problem. The reaction coefficient increases from left to right, while the diffusion coefficient increases from top to bottom. For each pair (x, c) , the color represents the first time at which the numerical concentration at x reaches a particular level c ; black curves denote equal activation times. All panels use the same time scale $t \in [0, 20]$.

order to observe the spurious solutions obtained for larger time steps. We therefore resort to the coordinate x_f of the right front of the solutions as a diagnostic reference, defined as the point where the profile of the curve crosses $c = 0.5$. All the runs are compared at the common time $T = 19.2$, the largest time that is a whole multiple of every step tested: $\Delta t = \{0.025, 0.05, 0.1, 0.2, 0.3, 0.4\}$. Figure 18 shows the outcome of the experiment. The qualitative observation of the reference is confirmed. Increasing the time step moves the front outward and at a certain point, the front disappears altogether: for $\Delta t \geq 0.2$ the interval is essentially full, so no $c = 0.5$ crossing exists at all. Large time steps do produce a spurious increase in numerical propagation, exactly as described. Quantitatively, the reading is truly delicate: the three time steps that admit a front give

$$x_f = \{0.6889, \quad 0.7268, \quad 0.8145\} \quad \text{for} \quad \Delta t = \{0.025, 0.05, 0.1\},$$

so the solution at $\Delta t = 0.1$ is not yet insensitive to further temporal refinement. This does not directly contradict the reference, since the criterion used there to assess convergence is not reported. Our numerical result only shows that, when the front position is used as a diagnostic, the solution still changes appreciably under further temporal refinement.

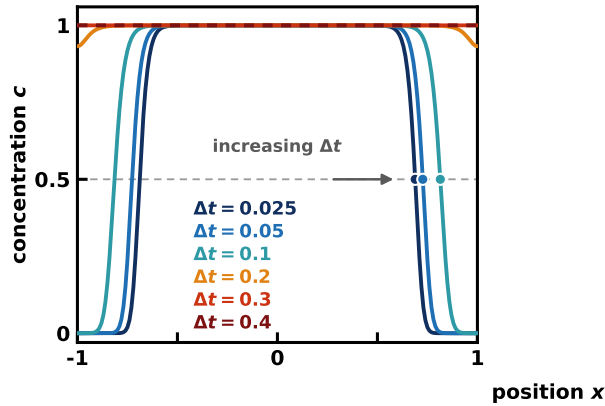


Figure 18: Time-step sensitivity plot at $\alpha = 2.0$, $d = 2 \cdot 10^{-4}$. All the runs are compared at the common time $T = 19.2$. Here, we show the six final profiles, with the $c = 0.5$ level marked by the dashed line and the resulting front positions by the circles. Only three runs cross that level. For $\Delta t \geq 0.2$ the interval is full and no crossing exists. The profile at $\Delta t = 0.4$ is dashed because it basically coincides with the one at $\Delta t = 0.3$. On the plateau $|x| \leq 0.5$ all six runs agree, so the disagreement is confined to the fronts.

4.2 The brain connectome as a metric graph

Having verified the nonlinear solver on a controlled one-dimensional problem, we now move to the network on which the disease is assumed to spread, reflecting the prion-like hypothesis that misfolded proteins propagate through the brain’s axonal fiber tracts. Following [9], we represent the brain as a weighted undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ whose vertices are anatomical regions and whose edges are the axonal bundles connecting them. The graph is extracted from the public Budapest Reference Connectome v3.0 [10], which aggregates the tractography of 418 healthy subjects of the Human Connectome Project. The downloaded fine-resolution graph contains 1015 parcels and 71604 connections. For each fine connection, the file records how many subjects exhibit that connection, together with aggregate statistics about its fiber count and length. As a first preprocessing step, we retain only fine connections occurring in at least five subjects. This reduces the number of edges from 71604 to 37477, while leaving all 1015 parcels in the graph. The five-subject threshold is not explicitly stated in [9]; rather, it is inferred from the public dataset as the unique integer threshold reproducing the 37477 fine edges reported in the reference. For comparison, thresholds of four and six subjects yield 40895 and 34718 edges, respectively. The resulting fine graph is then mapped onto the 83 anatomical regions of the FreeSurfer parcellation, as in [9]. Several fine parcels therefore collapse onto the same anatomical region. Among the 37477 retained fine connections, 7895 connect parcels belonging to the same final region and consequently become internal to an aggregated node. The remaining 29582 connections join distinct regions, but many connect different fine parcels belonging to the same pair of anatomical regions; so, such parallel connections are merged, resulting in the final graph with 83 vertices and 1130 edges.

Each retained fine connection f is assigned the weight $w_f = \frac{n_f}{\langle \ell_f \rangle}$, where n_f and $\langle \ell_f \rangle$ denote its statistically aggregated values of fiber count and length, respectively. When several fine connections join the same pair of anatomical regions I and J , their contributions are summed, resulting in the final weight A_{IJ} of a bundle.

Before any simulation is performed, the reconstructed connectome is verified against the graph statistics reported in [9]. Our results are reported in Table 4¹.

Quantity	Value
fine graph vertices / edges	1015 / 37477
aggregated vertices / edges	83 / 1130
degree range	6–48
mean fibre number	40.16
mean fibre length	38.40 mm
mean adjacency $\bar{A}_{IJ}$	1.5702
adjacency range	0.0085–35.3221
mean weighted degree $\bar{D}_{II}$	42.7547
weighted degree range	2.0505–127.6435

Table 4: Statistics of the reconstructed 83-region connectome. The degree of a region is the number of regions it connects to and its weighted degree $D_{II} = \sum_J A_{IJ}$ the total connectivity it carries. The fibre number of a connection is the number of fibre tracks that tractography reconstructs between its two regions, as a median over the cohort: the same holds for the fibre length. Here their average values over the connections are reported. Rounded to the precision at which [9] prints them, all the values coincide with the published ones.

The definition of the connectivity weight has an important consequence for the resulting network. Since each fine-edge contribution is proportional to a fiber count and inversely proportional to a fiber length, connections supported by many short fibers receive larger weights, whereas long-range connections are penalized. After aggregation, this effect is inherited by the regional adjacency A_{IJ} through the sum of all fine connections joining the same pair of regions. At the level of the anatomical groups used in this chapter, the total connectivity between the frontal and parietal groups is 174.8 and that between the parietal and occipital groups is 189.7. In contrast, the total frontal-temporal connectivity is only 0.85. Figures 19, 20 and 21 show the resulting computational domain.

Figure 22 summarizes the connectivity of the reconstructed graph through the degree of its vertices, their weighted degree and the adjacency matrix. The latter reflects the same organization described in [9], as we’re going to explain. The two dense diagonal blocks correspond to connections within the two hemispheres, whereas the off-diagonal blocks are much sparser and contain the interhemispheric connections. There are strong connections also within the four lobes, indicated along the diagonal of the matrix, drawn from the bottom-left to the top-right corner (4 lobes $\times$ 2 hemispheres). The coloured strips along the four sides identify the seven anatomical groups used in [8]. Neither these groups nor the four cortical lobes labels are part of the Budapest dataset, whose vertices carry their FreeSurfer anatomical names, the hemisphere and a cortical or subcortical flag, but no such anatomical grouping: both partitions (the four lobes classification of [9] and the seven classes of [8]) are obtained from the dataset anatomical names, through the name-based classification rules implemented in the solver. We resorted, in particular, to a `classify` function, which follows the standard Desikan–Killiany anatomy. We also observe that although the vertices retain their anatomical interpretation as identifiable brain regions, the coarse-grained edges do not possess a unique physical length after aggregation, since each of them represents the combined effect of several tractographic connections produced by the preprocessing rather than a single anatomical tract with a well-defined geometric length. We therefore assign a unit metric length, $h_e = 1$, to every edge and encode the tractographic information through the corresponding connectivity weight. The non-zero adjacency values span $0.0085 \leq A_{IJ} \leq 35.3221$, with a mean of 1.5702, reproducing the published range and mean to the reported precision. Within Fig. 22, panel (c), the most

¹Although [9] describes the edge weights in terms of fiber quantities averaged over the patients’ cohort, the published fine-graph statistics are reproduced by using the *median* fiber-count and length fields of the public dataset. Specifically, the reconstruction uses $w_f = \frac{n_f^{\text{median}}}{(\langle \ell_f \rangle)^{\text{median}}}$, followed by summation over parallel fine connections for the construction of the final graph. We therefore retain the median-based preprocessing, while treating this choice as a data-driven reconstruction rather than as a statement about the undocumented implementation used in the reference.

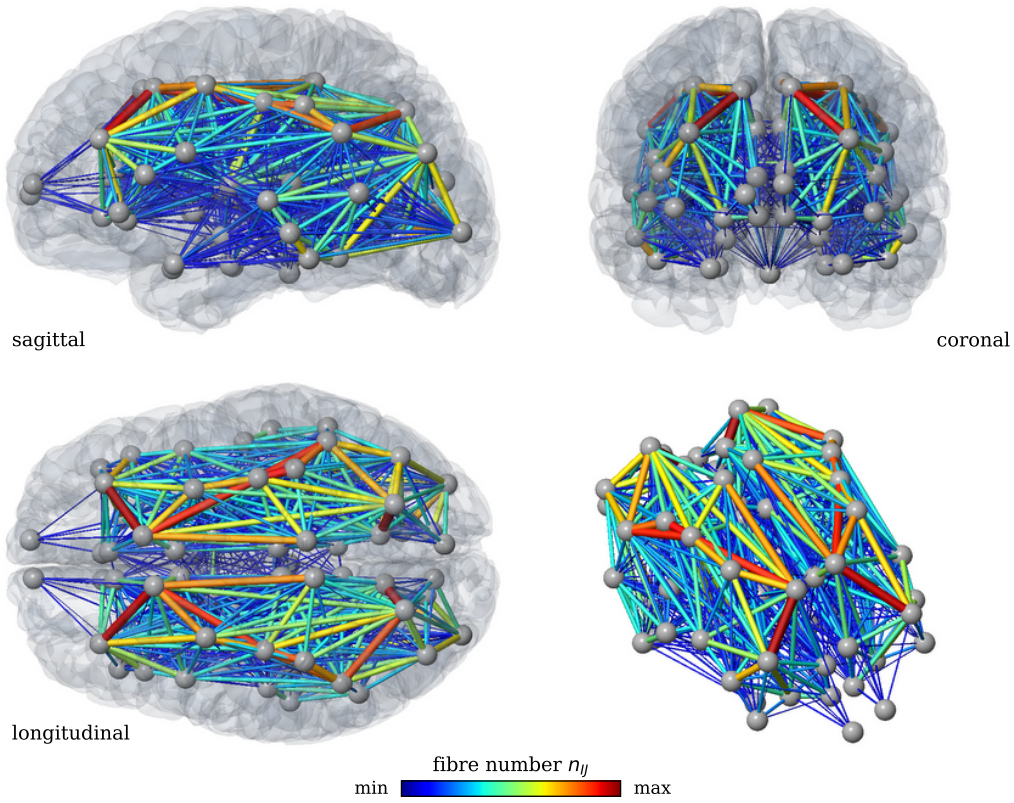


Figure 19: The connectome as a brain network: sagittal, coronal, longitudinal and oblique views of the 83 vertices and 1130 connections with and without the translucent pial surface. Colour and thickness of a connection both grow with its fibre number, from 1 to 595.5.

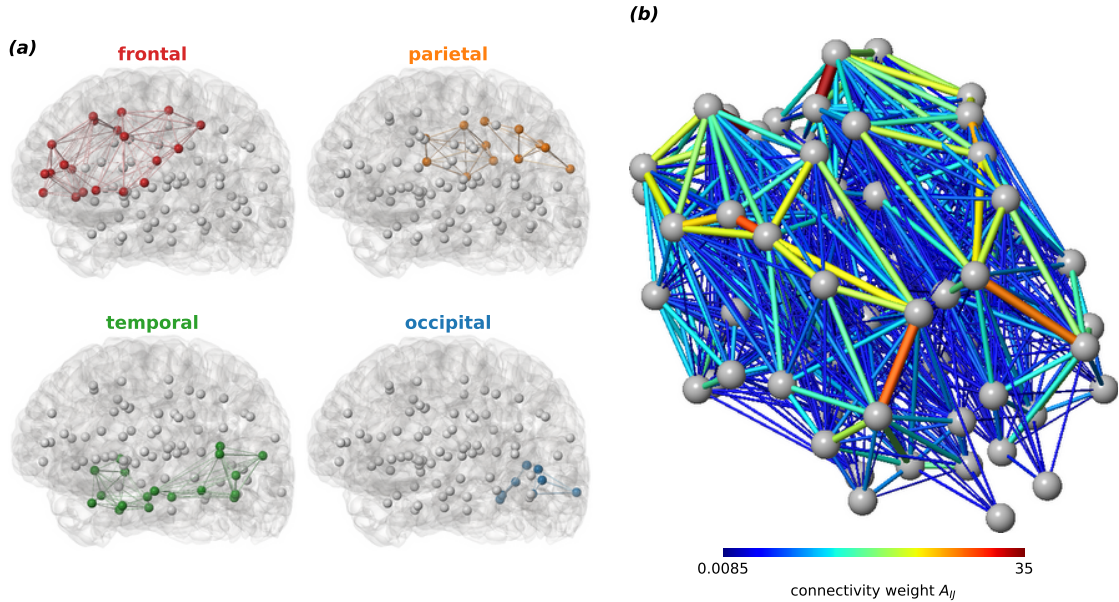


Figure 20: (a) The four cortical lobes: the regions of a lobe are coloured and the connections internal to it are drawn; white vertices belong to the remaining regions of a finer classification. (b) The resulting graph in the oblique view without the pial surface, connections coloured by connectivity weight. This is the abstract object the finite element method computes on; since an aggregated connection has no unique physical length, we choose for every edge a unit metric length: the anatomical information enters through its weight, the vertex coordinates being used for visualization only.

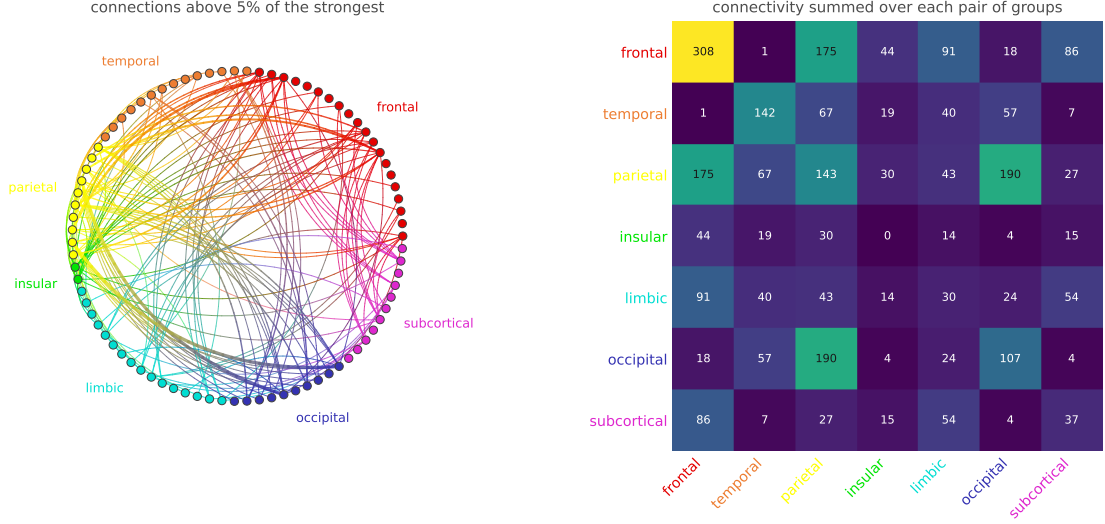


Figure 21: Connectivity of the reconstructed graph at the level of the seven anatomical groups. Left: the connections between regions of different groups with a weight above 5% of the strongest one, on a circular layout grouped by anatomical group, following [8]; each arc blends the colours of its two groups. Right: total connectivity between each pair of groups, measured as the sum of the weights A_{IJ} of all the connections joining a region of one group to a region of the other, the diagonal holding the connectivity internal to each group.

important feature that can be extracted is the pronounced heterogeneity of the network: most pairs of regions are either disconnected or only weakly connected, whereas a comparatively small number of connections carry much larger weights. Together with the heterogeneous weighted degree in panel (b), this reveals the presence of strongly connected hubs embedded in a much more weakly connected network. This structure is directly relevant to the spread of the disease. In the network model of [9], highly connected regions are expected to become effective hubs for the propagation of misfolded proteins: the disease does not propagate over a uniform network, but through a small number of preferentially connected pathways and hubs.

4.3 Network model and metric-graph formulation

We solve the Fisher–Kolmogorov equation posed on the (metric) graph presented in the previous section; on every edge e :

$$\partial_t c(s, t) - \partial_s [D_e \partial_s c(s, t)] = \alpha(s) c(s, t) [1 - c(s, t)]. \quad (10)$$

Up to the nonlinear right-hand side, this is the same parabolic problem studied in the first part of this work. The only new ingredient is the nonlinear reaction term. The references solve a simpler system, with one unknown per node and no spatial structure inside the connections. We will refer to the models of the references [8, 9] as the *nodal models*, that serve as the term of comparison next to our finite element solution. Each reproduced experiment uses the time scheme and way of treating the nonlinear term of its own reference. Table 5 summarizes the two schemes.

	Fully implicit [7, 9]	Semi-implicit [8]
Time discretization	Backward Euler ($\theta = 1$)	Crank–Nicolson ($\theta = 1/2$)
Nonlinear term	implicit, at t^{n+1}	semi-implicit, second-order extrapolation
Initialization	$\mathbf{c}^0$	$\mathbf{c}^{-1} = \mathbf{c}^0, \mathbf{c}^0$
Solve per step	Newton iteration	linear system, matrix rebuilt
Order in time	first	second

Table 5: The time discretization strategies adopted by the references, as implemented in this work. Both are based on the θ -method, combined with a different treatment of the nonlinear reaction term. In the semi-implicit scheme, each time step requires only a linear solve. The resulting matrix must however be rebuilt and refactorized at every step.

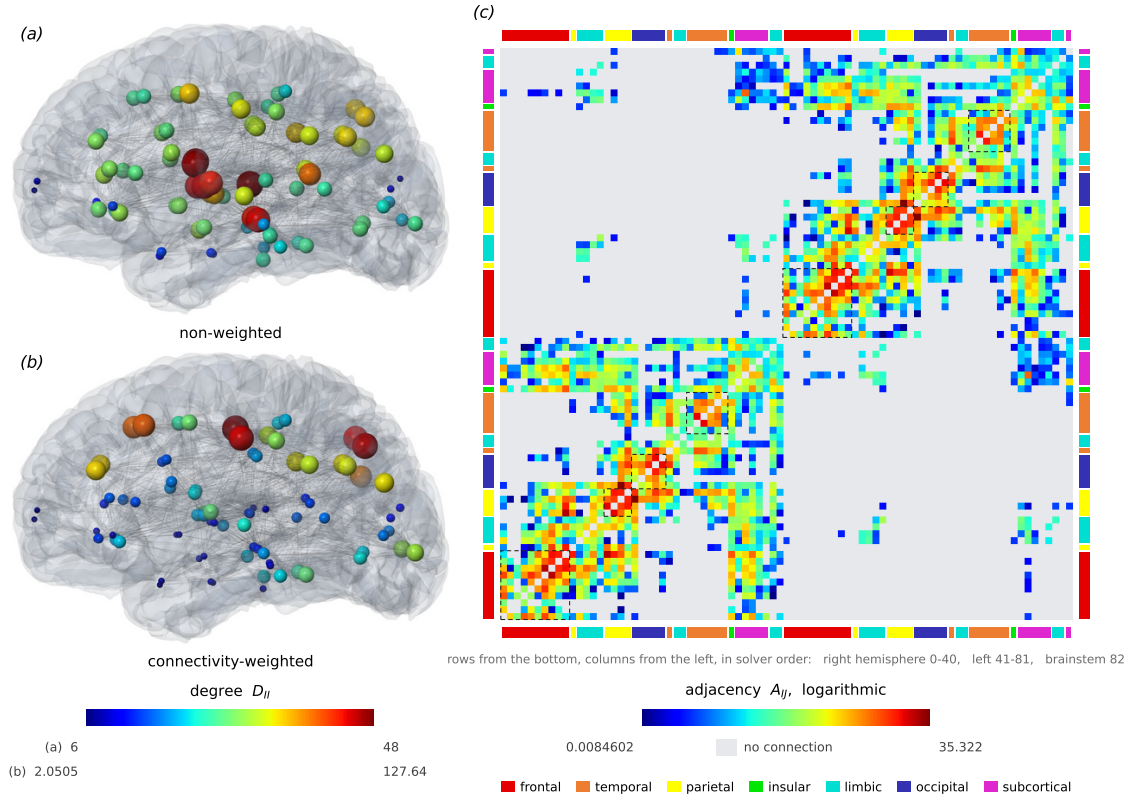


Figure 22: (a) Non-weighted degree and (b) connectivity-weighted degree; each panel is normalized to its own range. (c) Weighted adjacency matrix A_{IJ} displayed on a base-10 logarithmic scale over the complete non-zero range. A logarithmic colour scale is used to make both weak and strong connections visible within the same panel. Missing connections are shown separately in grey, while the coloured strips identify the seven anatomical groups. Rows and columns follow the solver ordering, with rows displayed from the bottom: right hemisphere 0–40, left hemisphere 41–81 and brainstem 82. The comparison of (a) and (b) shows that the regions with the most connections are not necessarily the hubs of the weighted network.

4.4 Comparison with the network model

We reproduce the baseline simulation of [9]: conversion rate $\alpha = 0.5$, misfolded protein seeded in the entorhinal cortex at $c_0 = 0.1$, all other regions healthy and 100 implicit steps of $\Delta t = 0.4$ years. Following the reference, the evolution is summarized by a biomarker: for a group of regions $\mathcal{I}$, the abnormality is the concentration averaged over its vertices and expressed as a percentage,

$$C_{\mathcal{I}}(t) = \frac{100}{|\mathcal{I}|} \sum_{I \in \mathcal{I}} c_I(t), \quad (11)$$

evaluated for the four cortical lobes and for the network as a whole. The simulation is run both with the nodal model, the formulation of the reference implemented literally and with the finite element model, here at one element per connection, so that the two systems share the same 83 unknowns and the same diffusion matrix and differ in the formulation alone. The biomarker curves have the expected sigmoid shape in both. The 50-percent crossing of the network average occurs at 11.11 years for the nodal model and at 12.68 years for the finite element model. The shift is expected: in the nodal model the domain consists of the 83 vertices only, whereas in the finite element model the connections are part of the domain as well and they are initially uninfected. The same seed, $c_0 = 0.1$ on the two entorhinal vertices, must therefore also fill the metric space between the vertices before their concentrations rise and the finite element curve shifts to the right by about 1.5 years. The two formulations describe different discrete systems, so their absolute crossing times are not expected to coincide. The activation sequence, however, is not reproduced by either: [9] report a separation of roughly five years between the temporal and the occipital lobe; our four lobe curves cross the 50-percent level within $3.9 \cdot 10^{-7}$ years of each other in the nodal model and $8.4 \cdot 10^{-3}$ years in the finite element model. The following section examines this discrepancy.

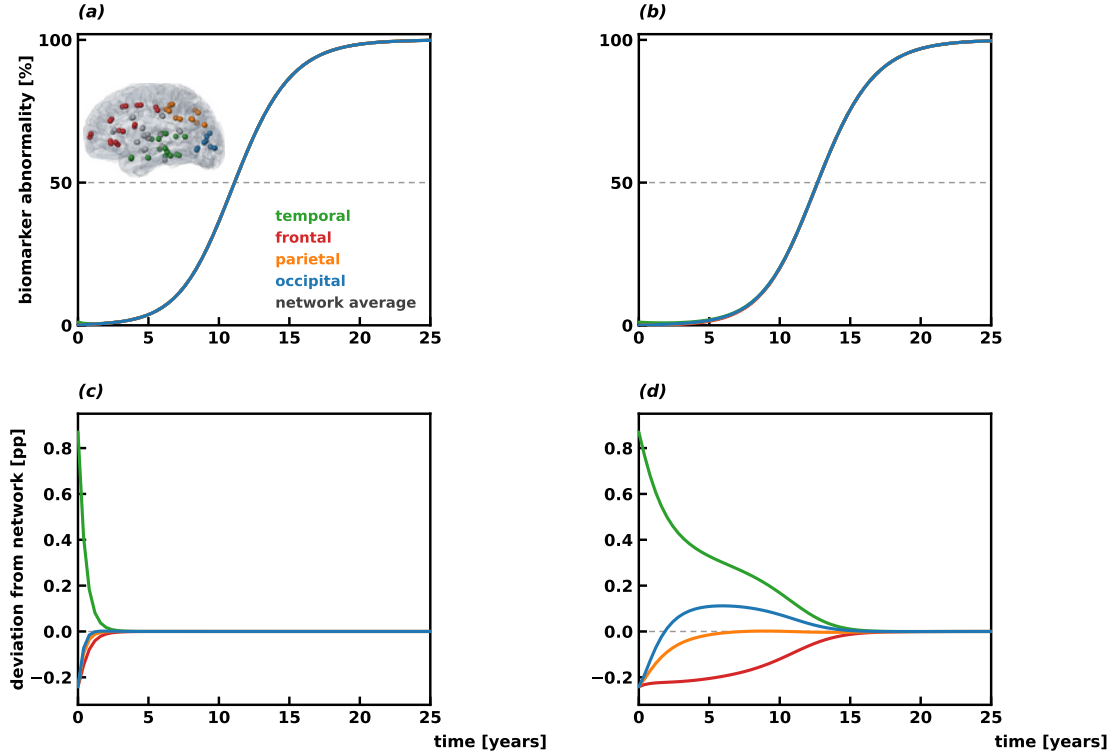


Figure 23: Biomarker abnormality of the four lobes and of the network, at $\alpha = 0.5$ with 100 steps of $\Delta t = 0.4$ years: (a) nodal reference, (b) metric-graph finite element model (one element per connection), with the four lobes in the colours of the inset. In both models the biomarker is the average of the concentrations at the 83 anatomical vertices, so the two curves start from the same value; the simulation runs to 40 years and the first 25 years, the curves having saturated by then. The four regional curves nearly coincide, crossing 50% within $3.9 \cdot 10^{-7}$ years of each other in (a) and $8.4 \cdot 10^{-3}$ years in (b), so the lower panels (c) and (d) report each curve minus the network average.

4.5 What controls the regional separation

Before explaining the discrepancy, it is useful to make some observations.

First, the graph. Although the reconstructed connectome reproduces the topology and global statistics reported in the reference, local differences in individual edge weights remain entirely possible, since part of the preprocessing pipeline had to be inferred from the public data rather than reproduced from a documented procedure.

Second, the discrepancy is not caused by the numerical model (as seen in the previous section) or by the definition of the lobes. In particular, the near-synchronization is already present at the level of the 83 individual regions: the biomarker concentrations all cross the 50% level within $6.4 \cdot 10^{-5}$ years in the nodal model and within 0.20 years in the finite element formulation, so no grouping of the nodes into lobes can separate the lobe curves by years. Nor is the reference's separation recovered by reasonable changes of the space and time discretization rules. Every tested combination reported in the repository show that these changes move the absolute timing of the curves, but not their near-synchronization.

Third, some implementation details remain unspecified: not only the complete graph-aggregation procedure, but also the precise implicit scheme adopted and the effective temporal scaling associated with the diffusion term. The reference defines the connectivity weights as $A_{IJ} = \frac{n_{IJ}}{\langle \ell_{IJ} \rangle}$, where the fibre count (n_{IJ}) is dimensionless and the fibre length (ℓ_{IJ}) is measured in millimetres. Therefore, if these quantities are interpreted dimensionally, $[A_{IJ}] = \text{mm}^{-1}$. On the other hand, the evolution equation is expressed in years, so that $[\frac{dc}{dt}] = \text{yr}^{-1}$. The reaction term is dimensionally consistent because the conversion rate satisfies $[\alpha] = \text{yr}^{-1}$. For the transport term, however, dimensional consistency would require an additional coefficient, say ρ , such that

$$\frac{dc}{dt} + \rho L c = \alpha c(1 - c),$$

with

$$[\rho] = \frac{\text{mm}}{\text{yr}},$$

so that $([\rho L] = \text{yr}^{-1})$. In this interpretation, ρ would set the temporal scale of transport on the network, analogously to how α sets the time scale of the reaction. Such a transport coefficient is not introduced explicitly in the published formulation. We noticed that the relative strength of network transport and local reaction strongly affects the temporal separation between the lobes. With the reaction switched off, the nodal model and the finite element model read

$$\frac{d\mathbf{c}}{dt} = -\rho L \mathbf{c} \quad \text{and} \quad \mathbf{M} \frac{d\mathbf{c}}{dt} = -\rho L \mathbf{c},$$

where L is the same matrix in the two, because at one element per connection the discrete Hamiltonian of Section 2.2 with $V = 0$ and diffusion coefficients $D_e = w_e$ coincides with the connectome Laplacian. What changes is the matrix in front of the time derivative.

In both cases the solution is a sum of exponentially decaying modes. The mode with zero rate is the uniform state the network tends to; every other mode decays at the rate $\rho \lambda_k$, where the λ_k are the eigenvalues of L in the nodal model and the eigenvalues of L relative to $\mathbf{M}$, $L\mathbf{v} = \lambda \mathbf{M}\mathbf{v}$, in the finite element one. The slowest non-zero rate decides how long a difference between regions can survive, so diffusion homogenizes the network on the time scale

$$\tau_{\text{diffusion}} \sim \frac{1}{\rho \lambda_2},$$

with λ_2 the smallest non-zero eigenvalue of the model at hand. Reaction alone makes the concentration grow at every vertex on the time scale

$$\tau_{\text{reaction}} \sim \frac{1}{\alpha}.$$

The two processes compete and their ratio defines the Damköhler number

$$\text{Da} = \frac{\tau_{\text{diffusion}}}{\tau_{\text{reaction}}} = \frac{\alpha}{\rho \lambda_2}. \quad (12)$$

When $Da \ll 1$ the network homogenizes faster than the reaction grows and strong regional synchronization is expected; when Da is large the differences created by the seed survive and the lobes' abnormality curves can separate.

For the reconstructed connectome the nodal model has $\lambda_2 = 0.7723$. In the finite element model, the same diffusive term ρLc is filtered through the mass matrix $\mathbf{M}$, so the rate of change is governed by $\mathbf{M}^{-1}L$ rather than by L alone. With mass lumping (see Section 4.5.1 for the description of the mass lumping as a stabilization technique) the generalized spectrum changes again; in this connectome the second eigenvalue decreases from 0.0635 (the value of the consistent mass) to 0.0606, so the slowest diffusive mode becomes slightly slower. This decrease is not obvious a priori and has to be determined from the generalized eigenvalue problem. So, in the finite element model, the smallest non-zero eigenvalue is twelve to thirteen times smaller than 0.7723. With $\rho = 1$ and $\alpha = 0.5$, the nodal model therefore has $Da = 0.65$, while the finite element model has $Da = 7.9$. At this point, however, the reasoning seems to break down: despite $Da = 7.9$, the lobe-wise curves remain nearly synchronized. This suggests that the slowest diffusive mode of the whole network is not necessarily the one that controls the separation between lobes. For this reason, we also consider the eigenvectors associated with the spectrum containing λ_2 . Each eigenvector describes a different spatial pattern of concentration differences that diffusion tends to smooth out, while the corresponding eigenvalue determines how fast that pattern decays. We find that the mode associated with λ_2 does not represent a contrast between anatomical lobes. Therefore, for the lobe-wise biomarker, the relevant quantity is not the slowest eigenvalue of the entire network, but the slowest relaxation rate among the modes that actually describe differences between lobes.

The eigenvector associated with λ_2 confirms this: its largest components are concentrated on the frontal and temporal poles and on the entorhinal cortices, with opposite signs in the two hemispheres. Therefore, the slowest diffusive mode of the whole network represents mainly an inter-hemispheric peripheral imbalance rather than a contrast between anatomical lobes.

The decay rates, for the nodal model, range from 10.0 to 49.4, with the temporal-occipital contrast, for instance, relaxing at a rate of 21.6. For the finite element model, the slowest decay rate is 1.11 with the consistent mass matrix and 0.86 with the lumped version. With these rates, we can define a lobe-scale Damköhler number,

$$Da_{\text{lobe}} = \frac{\alpha}{\rho \lambda_{\text{lobe}}}.$$

At $\rho = 1$, it is 0.05 for the nodal model and 0.45 and 0.58 for the finite element model, with the consistent and the diagonalized mass matrix respectively. In every case we have a $Da_{\text{lobe}} < 1$, so at the lobe-scale, transport still beats reaction and the synchronized curves are exactly what this number predicts. Figures 24 and 25 quantitatively confirm the reading: the curves' separation becomes appreciable as Da_{lobe} crosses one.

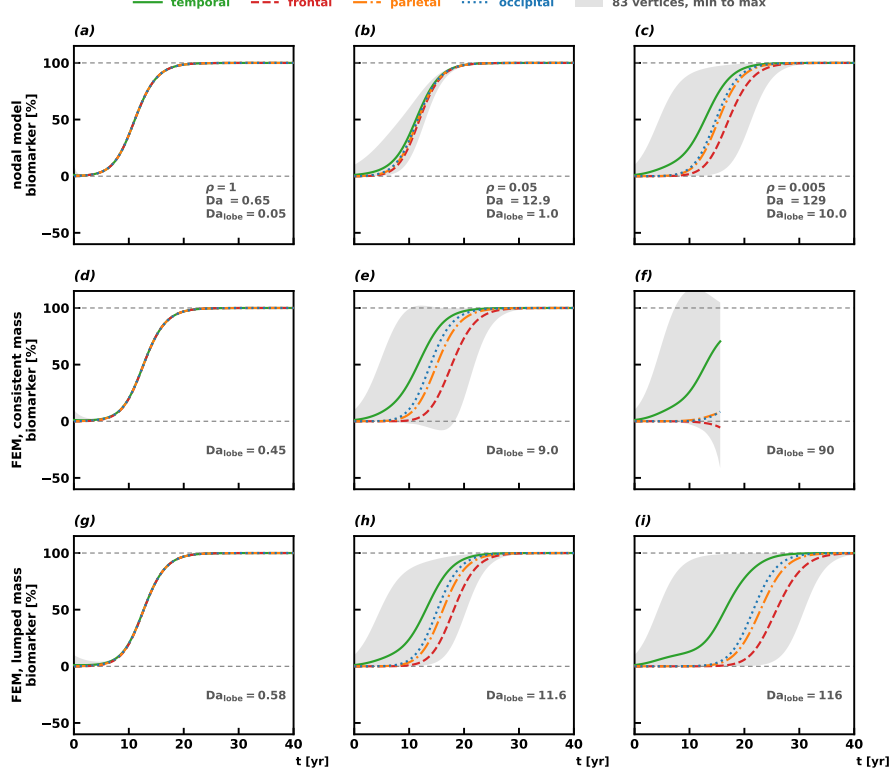


Figure 24: Regional separation as a function of the Damköhler number (12), for the three models, at one element per connection with the fully implicit scheme, $\Delta t = 0.4$ years, the entorhinal seed and $\rho = 1, 0.05$ and 0.005 (columns). Rows: (a)–(c) the nodal model of the references; (d)–(f) the finite element model with the consistent mass matrix of the first part of this work; (g)–(i) the same model with the lumped mass matrix (see Section 4.5.1). Each panel shows the four lobe biomarkers and, shaded, the envelope of the 83 vertex concentrations; the dashed lines mark 0 and 100 percent. The lobes separate as Da grows in every model. With the consistent mass, the envelope leaves the physical range at $Da = 12.9$ (e) and at $Da = 129$ (f), where the Newton iteration fails at $t = 15.6$ years and so the run stops. With the lumped mass, the solution stays within $[0, 1]$ in every panel.

Since the conversion rate is uniform, the different lobe activation times cannot be explained by α and are actually determined by the position of the seed (here at the temporal lobe) and by the network connectivity. The total temporal connectivity, in particular, suggests the ordering parietal–occipital–frontal, with weights 75.0, 55.6 and 1.0, respectively. However, the biomarker is averaged over all regions of each lobe. Once the connectivity is considered per region, the temporal–occipital coupling is stronger than the temporal–parietal one (7.0 versus 6.3 on average). The resulting ordering is therefore temporal–occipital–parietal–frontal.

We conclude that:

- The relative strength of reaction and transport, quantified by the Damköhler analysis, controls *how much* the lobe curves separate in time (see panel (b) of Figure 25).
- For a fixed seed and a uniform conversion rate, the spatial distribution of the connections of the reconstructed connectome controls *in which order* the lobes cross the biomarker threshold.

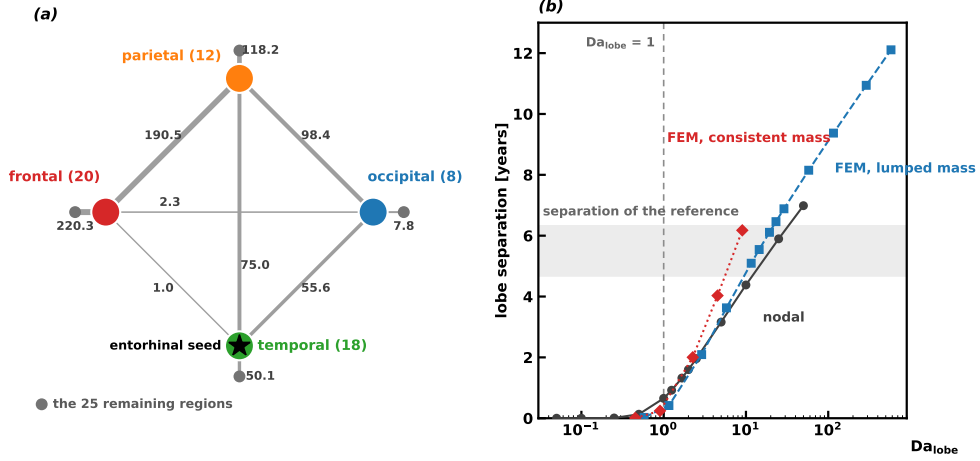


Figure 25: (a) The connectome compressed over the four lobes of the biomarker. (b) Lobe separation against Da_{lobe} for the three models; the dashed line marks $Da_{\text{lobe}} = 1$, below which every model keeps the lobes together and the grey band is the separation of the reference, 5.5 years read from figure 7 of [9] with a margin of 15%.

4.5.1 Mass lumping as a stabilization technique

With the finite element model, using one element per connection and the fully implicit time scheme, we observe that at low and moderate Damköhler numbers the solution may temporarily leave the admissible interval $[0, 1]$, but it eventually recovers. At $Da = 0.65$ the minimum concentration reaches only $-1.3 \cdot 10^{-4}$, while at $Da = 12.9$ it reaches -0.08 and the maximum reaches 1.02 . From the next tested scaling, $Da = 16.2$, the solution no longer recovers. At $Da = 129$ the concentrations reach -0.41 and 1.17 by $t = 15.6$ years, after which the nonlinear solver cannot complete the next time step. The same qualitative behaviour is observed with the semi-implicit scheme of Table 5 and the failure persists under mesh refinement, indicating that it is not simply a coarse-mesh effect. The origin of this behaviour can be understood from the finite element system

$$\mathbf{M} \frac{\mathbf{c}^n - \mathbf{c}^{n-1}}{\Delta t} + \mathbf{H} \mathbf{c}^n = \mathbf{f}(\mathbf{c}^n),$$

where $\mathbf{H}$ is the discrete Hamiltonian of Section 2.2 with $V = 0$ and diffusion coefficients $D_e = \rho w_e$, so that $\mathbf{H} = \rho L$ at one element per connection. When transport is weak, the diffusion term becomes small and the update is approximately governed by

$$\mathbf{c}^n - \mathbf{c}^{n-1} \approx \Delta t \mathbf{M}^{-1} \mathbf{f}(\mathbf{c}^n).$$

With the consistent mass matrix, $\mathbf{M}^{-1}$ contains negative off-diagonal entries. This means that, through $\mathbf{M}^{-1}$, the reaction computed at one vertex can contribute with the opposite sign to the increment of neighbouring vertices, producing small negative concentrations. Once a concentration crosses below zero, the logistic term reinforces the error because

$$c < 0 \quad \implies \quad c(1 - c) < 0.$$

The instability therefore develops in two stages: the consistent mass discretization first generates an undershoot and the nonlinear reaction then amplifies it. When transport is strong, diffusion smooths these small excursions and the solution returns to the physical interval; when transport becomes too weak, it can no longer compensate for them.

The Newton failure observed at $Da = 129$ is therefore a consequence rather than the cause of the instability. By $t = 15.6$ years the solution has already reached $c_{\min} \simeq -0.41$ and the logistic reaction tends to drive the negative concentrations further down. Since the reaction is evaluated at the new time level, the fully implicit scheme requires the solution of a nonlinear system at every time step. Newton's method solves this system iteratively by correcting a current approximation until the residual is sufficiently small. In our implementation, the Newton iteration is damped and equipped with a safeguard that prevents the intermediate iterates from leaving the interval $[-0.5, 1.5]$. At the step following $t = 15.6$ years, the corrections required by the nonlinear system tend to push some

concentrations below the lower bound. The damping can reduce these corrections, but eventually the solver cannot both remain inside the safeguard and reduce the residual to the required tolerance, so the iteration stops. The semi-implicit scheme, which solves a linear system at every step and has no such safeguard, continues at the same scaling and eventually diverges to $[-507, 523]$. This confirms that Newton is not creating the instability: it is only the point at which the fully implicit computation can no longer follow an already nonphysical solution.

The same loss of positivity is reproduced in the one-dimensional problem of Section 4.1, showing that the mechanism is not specific to the connectome topology.

A standard remedy for this loss of positivity is *mass lumping*. The consistent mass matrix is replaced by the diagonal matrix obtained from its row sums, while the reaction integral is evaluated using the vertex rule²,

$$f_i = M_{ii}^L \alpha_i c_i (1 - c_i).$$

Together, mass lumping and the vertex rule make the reaction treatment local: the lumped mass removes the cross-node coupling introduced by $\mathbf{M}^{-1}$, while the vertex rule makes f_i depend only on the local concentration c_i . The reaction therefore takes the same node-wise form as in the nodal model, while the transport operator remains the finite element one.

We added mass lumping as an option in the finite element library and repeated the sweep under the same conditions. With the consistent mass, the concentration envelope already leaves $[0, 1]$ at $\text{Da} = 12.9$ and the computation eventually breaks down at $\text{Da} = 129$. With the lumped mass, instead, all vertex concentrations remain within $[0, 1]$ throughout the sweep, while the relevant physical behaviour is preserved: the lobe curves progressively separate as transport is weakened, as discussed in the previous paragraph.

At the original transport scale, the two formulations are almost indistinguishable, with the network crossing time changing only from 12.68 to 12.72 years. This shows that mass lumping stabilizes the finite element model in the weak-transport regime without significantly modifying the solution where the consistent formulation is already reliable. For this reason, in the rest of the chapter we use the consistent mass where transport is strong and the lumped mass where transport is weak, as in the seeding experiment of Section 4.6. Every result from this point onward is therefore a finite element result.

4.6 Tau seeding and expected progression

The two hallmark proteins of Alzheimer’s disease are tau and amyloid- β . Amyloid- β is not studied here: the connectome encodes axonal pathways, which are the propagation routes of intracellular tau, whereas amyloid- β spreads extracellularly, which is why the reference itself simulates only tau on this graph [9]. Tau enters the model through the initial spatial location alone: inclusions appear first in the transentorhinal region and the expected progression is from the entorhinal cortex to the temporal lobe and then to the interconnected neocortex [7]. On the graph, the seeding is the entorhinal vertices at $c_0 = 0.1$.

Figure 26 shows the computed progression below its clinical counterpart, namely the staging drawings of [7]. Three representative stages are selected as the first instants at which the network mean concentration reaches 10%, 40% and 80%. The solution is the finite element one with lumped mass and the fully implicit scheme, at $\rho = 0.005$, corresponding to $\text{Da} = 129$ and $\text{Da}_{\text{lobe}} = 116$.

This transport scaling is essential for the progression to become visible. At $\rho = 1$, the scale used in the references, transport is sufficiently fast to synchronize the network: in a control simulation the four lobes cross the 50% level within 0.03 years of each other and the three stages would appear almost spatially uniform. The choice $\rho = 0.005$ should therefore be interpreted as a calibration of the transport time scale against the reference progression, rather than as an additional biological input. The only biological input of the simulation is then the location of the initial seed.

²The vertex rule approximates an integral over an edge $e = [i, j]$ of length h_e using only the values at its endpoints. Applied to the reaction term,

$$\int_e \alpha c_h (1 - c_h) \phi_i \, ds \approx \frac{h_e}{2} [\alpha c_i (1 - c_i) \phi_i(i) + \alpha c_j (1 - c_j) \phi_i(j)].$$

Since $\phi_i(i) = 1$ and $\phi_i(j) = 0$, this reduces to

$$\frac{h_e}{2} \alpha c_i (1 - c_i).$$

After summing over all edges incident to vertex i , one obtains $f_i = M_{ii}^L \alpha c_i (1 - c_i)$, so the reaction at vertex i depends only on the local concentration c_i .

What Figure 26 recovers has to be stated carefully, because the first station of the expected pattern is prescribed by the seed rather than produced by the model and because every run of Eq. (10) with a positive reaction rate ends with the whole network at $c = 1$. For the tau protein, the model produces one step of the expected sequence: from the entorhinal seed the pathology reaches the temporal lobe first, as the clinical description says. It does not produce the rest. The remaining lobes follow in the order of the graph-connectivity, so the occipital lobe, which the sequence of [9] leaves for last, is here the second to be involved and the frontal lobe, which the reference places immediately after the temporal one, is here the last, according to our Figure 24. In summary, the computed row starts from the entorhinal seed and fills the temporal lobe first; the spreading then moves posteriorly, the occipital lobe being the second of the four to cross the 50% level, before progressively involving the rest of the network.

The same caveat discussed in Section 4.5 remains valid. Within the later tau stages, the relative ordering of the cortical lobes is determined by the connectivity of the reconstructed graph rather than by the clinical staging sequence. The four lobes cross the 50% level in the order temporal, occipital, parietal and frontal, reproducing the ordering already observed in Figure 24. In our simulations, the frontal group is delayed by its weak effective connectivity, with the frontal pole being the least connected region of the graph. Since the conversion rate is uniform in the present model, regional differences in the progression are driven by connectivity alone. This motivates the introduction of region-dependent conversion rates, as in Section 4.7, where additional biological heterogeneity is included in the reaction term.

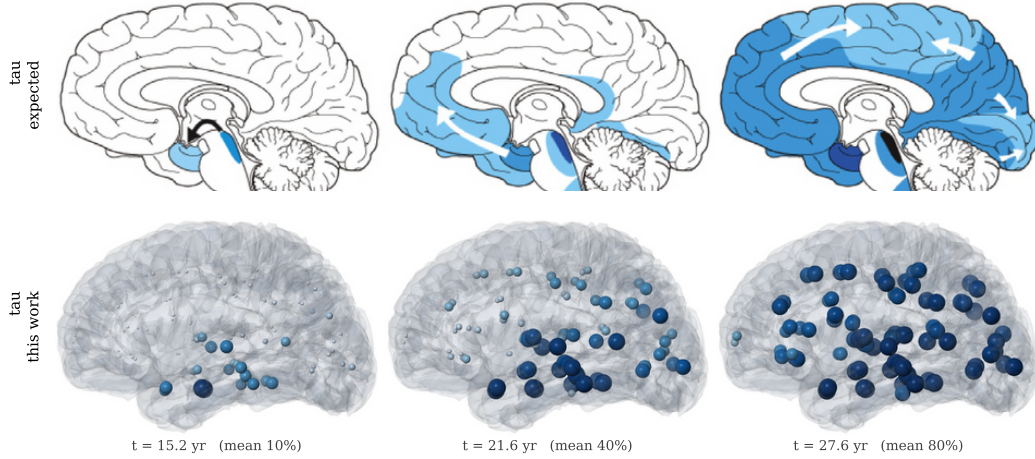


Figure 26: Expected and computed progression for tau. First row: clinical staging drawings from Figure 1 of [7], medial view. Second row: the progression computed in this section, with sphere radius and blue intensity growing with the local concentration, at the first instants at which the network mean reaches 10%, 40% and 80%. The finite element model uses lumped mass, $\rho = 0.005$, $Da = 129$ and $Da_{lobe} = 116$. Tau, seeded at the entorhinal cortex, first involves the temporal neighbourhood and then progressively the rest of the network. The three stages highlight the origin, direction and endpoint of the progression, determined by the network connectivity, as discussed in the text. At the second computed stage the temporal lobe has reached 86% and the frontal lobe 13%, with the occipital and parietal lobes in between at 49% and 36%, so the computed row shows the connectivity order and not the clinical one, whose second station is the frontal lobe. This motivates the introduction of further biology through heterogeneous reaction rates.

4.7 Deterministic model with regional reaction rates

The previous sections showed what can be explained by the connectome alone. With a uniform conversion rate, the network connectivity strongly influences the ordering of the lobe-wise curves. However, this ordering does not fully reproduce the clinical one. We now introduce the biological heterogeneity that was absent from the previous model by allowing the conversion rate to vary between anatomical regions.

We follow [8], who solve a patient-specific stochastic Fisher–Kolmogorov model on a 246-node Brainnetome graph and estimate the reaction coefficient region by region from a PET acquisition. Their formulation is nodal, with one unknown per vertex and a discrete graph Laplacian, whereas here we solve the corresponding deterministic problem with the metric-graph finite element model developed in this work. We do not attempt to reproduce the clinical trajectory of their patient.

Instead, we transfer their estimated mean regional reaction rates to our 83-region connectome and ask whether they change the ordering of the four lobes. Making α region-dependent therefore introduces the first biological heterogeneity in the model beyond the connectivity itself. The reaction coefficient is taken piecewise constant over seven anatomical groups, using the mean values reported in Table 1 of [8]:

$$\begin{aligned} \alpha_{\text{frontal}} &= 0.1801, & \alpha_{\text{temporal}} &= 0.1421, & \alpha_{\text{parietal}} &= 0.0627, \\ \alpha_{\text{insular}} &= 0.1005, & \alpha_{\text{limbic}} &= 0.1351, & \alpha_{\text{occipital}} &= 0.0545, & \alpha_{\text{subcortical}} &= 0.1147. \end{aligned}$$

The experiment is the staging comparison of Section 4.6 repeated with these rates and nothing else changed: the same entorhinal seeding, the same scaling $\rho = 0.005$, the same lumped mass and the same three-stage rule, so that the solutions differ from the uniform ones only through the reaction coefficients. The seven rates are rescaled to the vertex mean 0.5 of the uniform run, which preserves the reaction budget, $\text{Da} = 129$ and $\text{Da}_{\text{lobe}} = 116$ and only redistributes the rate; the horizon is extended to 80 years for the slow groups. The rates of the insular, limbic and subcortical regions enter the model in the same way, so the dynamics of the four lobes is shaped also by what happens around them; the ordering we compare is the one of the four lobes alone, against the expected pattern of Section 4.6.

Figure 27 shows the outcome. The tau lobes now cross the 50% level at 14.7, 19.9, 29.7 and 32.4 years, in the order temporal, frontal, parietal and occipital, which is the clinical sequence the uniform rate could not produce; the lower panel shows the same run through the biomarker curves of the four lobes. The comparison therefore suggests a division of roles rather than a strict separation: connectivity governs the transport pathways and contributes to the ordering, while the heterogeneous reaction field provides the dominant regional bias in this experiment.

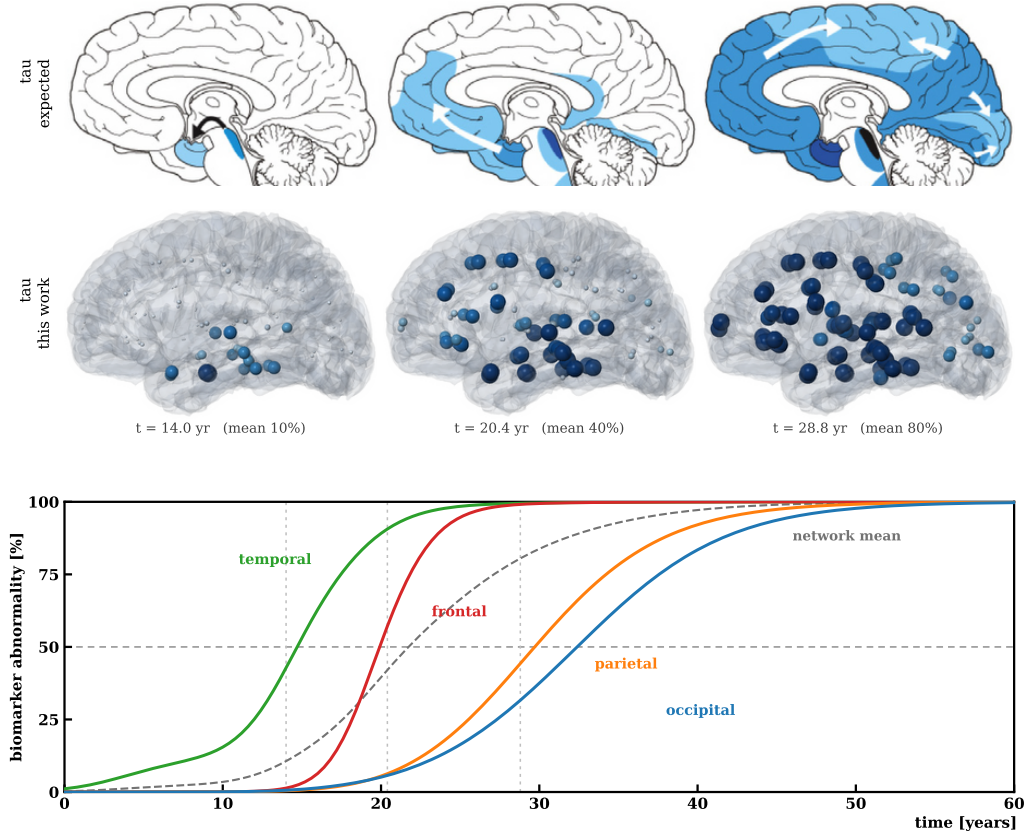


Figure 27: The experiment of Figure 26 repeated with the regional reaction rates of this section: same seed, same scaling $\rho = 0.005$, same lumped mass and the same three-stage rule, over 80 years. The computed disease progression now reaches the frontal lobe second and the occipital lobe last, which is the clinical sequence. Bottom row: the four lobe biomarkers of the same run, with the network mean dashed, the 50% level marked and the three stage instants as vertical lines, at 14.0, 20.4 and 28.8 years.

5 Performance analysis

5.1 Sequential baseline

The last numerical simulation is the reference problem for the performance analysis of this chapter. Timings are taken in a separate release build with visualization and diagnostic output disabled, a thread count equal to one, and reported as the median of five runs, after one warm-up.

Elements per connection	Unknowns	Matrix nonzeros	100 steps [s]	Per step [s]
1	83	2343	0.0832	0.00083
2	1213	5733	1.8694	0.01869
4	3473	12513	1.5486	0.01549
8	7993	26073	2.2343	0.02234

Table 6: Sequential baseline for the deterministic model. “100 steps” is the total time required to advance 100 semi-implicit time steps, while “Per step” is the corresponding average cost of one step. Each time step includes the update of the finite element system matrix, let us call it K^n , its numerical factorisation and the subsequent linear solve. One-time costs associated with input reading and the initial assembly are measured separately and account for less than one percent of the total execution time in every refined case.

In every refined case, almost the entire execution time is spent inside the time loop, while the one-time costs associated with input reading and initial finite element assembly account for only a small fraction of the total. Each iteration step involves the construction, factorization and solution of the linear system generated by the semi-implicit scheme. We notice that the cost per step does not increase monotonically with the number of degrees of freedom. In particular, the mesh with 1213 unknowns is slower than the one with 3473. This indicates that the matrix dimension alone does not explain the computational costs. Since every time step involves an algorithm for sparse factorization (SparseLU), the way in which the unknowns are ordered and consequently the amount of fill-in generated during the factorization may play a very important role.

For scale, [9] report that their nodal model runs its 100 implicit steps in 0.55 s without output visualization on a standard laptop; our metric-graph formulation at the same 83 unknowns takes 0.083 s for its 100 semi-implicit steps. The machines and the schemes differ, so this is an order-of-magnitude check rather than a direct comparison. It nevertheless shows that, at the coarsest level, introducing the metric-graph finite element formulation does not produce an excessive computational cost, while allowing the connections to be refined when required.

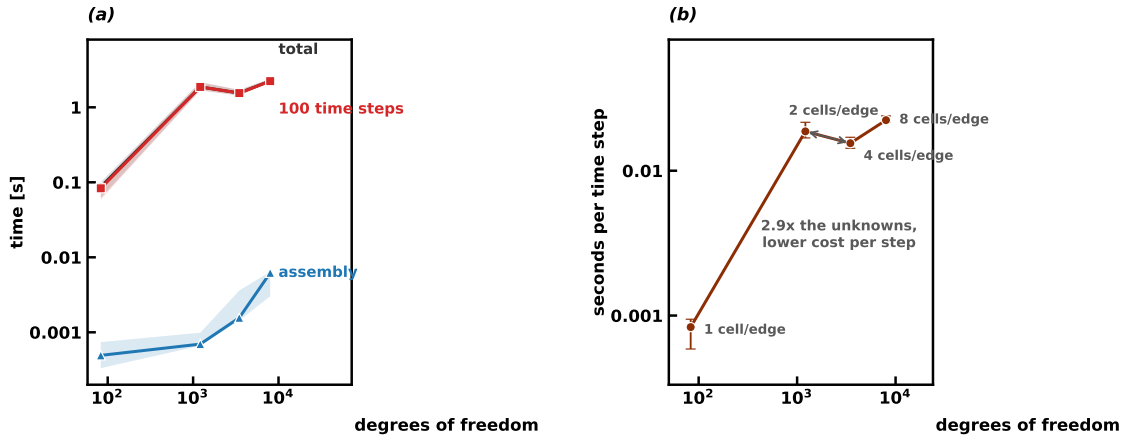


Figure 28: Sequential baseline. (a) assembly, time loop and total against the number of unknowns, with the band spanning the minimum and maximum of five runs; the total is hidden by the time loop (red line), which accounts for 95.9% of it on the coarsest mesh and for more than 99% on the refined ones, so the two curves overlap. (b) average cost per time step: the two-element mesh costs more than the four-element one.

The non-monotonic behavior observed in Figure 28 motivates the first question of the performance study: before parallelizing the solver, can the sequential sparse linear algebra itself be improved?

5.2 Matrix ordering and the sequential solver

With the semi-implicit scheme of Table 5, every time step advances the concentration by solving one linear system, which reads as follows. Writing M for the mass matrix, H for the discrete Hamiltonian presented in Section 2.2 and $R(v)$ for the reaction matrix assembled with the weight $\alpha(1 - v)$, the Crank–Nicolson update with the second-order extrapolation $\hat{c}^{n+1} = \frac{3}{2}c^n - \frac{1}{2}c^{n-1}$ is:

$$K^n c^{n+1} = b^n, \quad K^n = M + \frac{\Delta t}{2}H - \frac{\Delta t}{2}R(\hat{c}^{n+1}), \quad (13)$$

with

$$b^n = \left[M - \frac{\Delta t}{2}H + \frac{\Delta t}{2}R(\hat{c}^{n+1}) \right] c^n.$$

The graph topology and the finite element mesh do not change during the simulation. Therefore, the positions of the nonzero entries of K^n remain the same at every step. The numerical values of some entries, however, do change because the reaction term depends on the extrapolated concentration. For this reason the numerical factorization must be performed at every time step.

In the implementation used in the previous chapters, this system is solved with Eigen’s sparse LU factorization. Before the factorization, Eigen applies the default COLAMD ordering. The purpose of such an ordering is to reduce *fill-in*, namely the appearance of new nonzero entries in the factors even when the corresponding entries in the original matrix are zero. Since more fill-in implies more memory and more arithmetic operations, a poor ordering can make a sparse factorization expensive even when the matrix itself is relatively small. The unusual timings of Table 6 therefore suggest that Eigen’s default ordering may not be well suited to the structure of our finite element matrix. To test this hypothesis, we replay the same time loop, with the same matrices and right-hand sides, using different combinations of ordering and factorization. We consider the LU factorization with COLAMD, LU with the natural ordering that we introduced earlier in this work, LU with the approximate minimum degree ordering (AMD), LU after a reverse Cuthill–McKee (RCM) permutation and, exploiting matrix symmetry, the LDL^T factorization with AMD and with the natural ordering.

The natural ordering is particularly relevant because it is already built into the finite element implementation. As introduced in Section 1.2, the interior nodes are numbered consecutively connection by connection, while the 83 original graph vertices are numbered last. This has a direct consequence for the matrix structure. Interior nodes of one connection interact only with neighbouring nodes of the same connection and with its two endpoint vertices. The corresponding interior part of the matrix is therefore formed by independent tridiagonal blocks, one for each connection. The coupling between different connections appears only through the original graph vertices grouped at the end of the numbering. In other words, the natural numbering already separates the local one-dimensional structure of the connections from the small global interface formed by the 83 graph vertices. Eliminating the unknowns in this order first processes the interior of each connection and only afterwards reaches the common vertices.

Ordering	2 elements, $n_{dofs} = 1213$		8 elements, $n_{dofs} = 7993$		64 elements, $n_{dofs} = 71273$	
	factor nonzeros	step [ms]	factor nonzeros	step [ms]	factor nonzeros	step [ms]
LU, COLAMD	182208	26.84	104934	21.16	484614	140.31
LU, natural	10522	2.35	51202	10.43	430882	81.38
LU, RCM	80570	8.13	1103850	162.93	11594218	2013.1
LDLT, natural	5261	0.61	25601	1.93	215441	21.04

Table 7: Ordering study on the connectome meshes with 2, 8 and 64 elements per connection: nonzeros of the computed factors and representative per-step cost over 100 semi-implicit time steps. The reported cost is obtained as the sum of the median times measured for matrix rebuild, factorization and linear solve within the same benchmark run, making the measurement less sensitive to fluctuations. The reported timings are therefore intended for comparisons within this ordering study and should not be compared directly with the wall-clock times reported in Table 6, which were obtained in a different benchmark session.

These results explain the anomaly of the sequential baseline: on the two-element mesh, COLAMD produces 182208 nonzeros in the factors, more than it produces on the larger four-element mesh. The smaller problem is therefore more expensive because the chosen ordering generates more fill-in. The non-monotonic timing is thus an ordering effect rather than an intrinsic property of the discretisation size. Moreover, the RCM experiment gives another useful observation. RCM strongly reduces the matrix bandwidth, for example from 7989 to 937 on the eight-element mesh³. This

³The matrix bandwidth measures how far the nonzero entries extend from the main diagonal. For a nonzero entry

does not lead to a cheaper factorization. On the contrary, for this particular matrix structure it disrupts the favourable separation between connection interiors and graph vertices and produces substantially more fill-in. Indeed, at 64 elements per connection the resulting factors contain way more nonzeros than those obtained with the natural ordering.

Finally, since K^n is symmetric, we can replace the general LU factorization by a LDL^T factorization. The combination of the natural ordering and LDL^T gives the best performance among the tested configurations. At 64 elements per connection, the cost decreases from 140.3 ms per step to 21.0 ms.

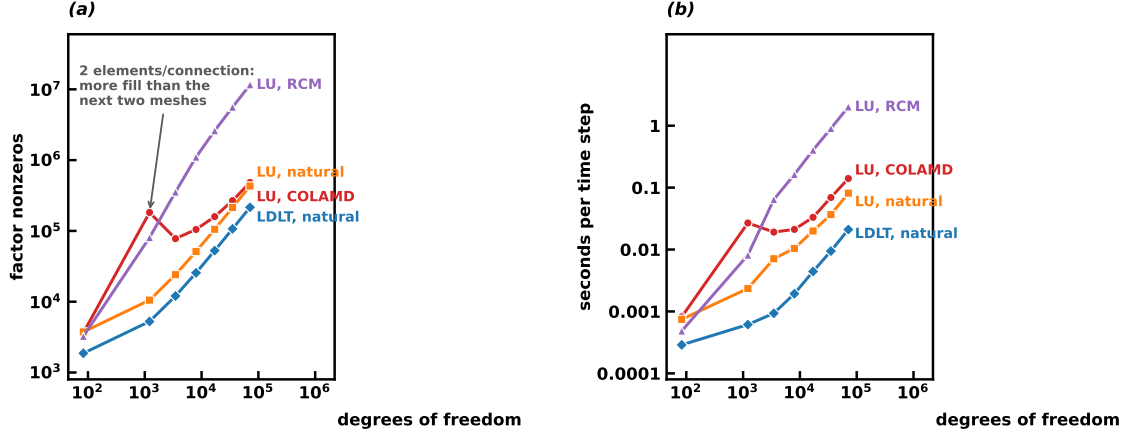


Figure 29: The ordering study. (a) factor nonzeros against the number of unknowns for the four orderings of Table 7; the marked point is the two-element mesh, on which the production ordering fills more than on the next two meshes. (b) median cost per time step. The LDL^T factorisation on the natural ordering is the sequential reference adopted in the following analysis.

Correcting the ordering and exploiting symmetry also change the location of the computational bottleneck. At 64 elements per connection, the factorisation now costs 6.1 ms and the triangular solves required by LDL^T 0.9 ms, whereas building the global sparse matrix costs about 14.0 ms per step. This suggests the next question. The natural ordering has already shown that the interior unknowns of each connection can be eliminated independently. Is it therefore necessary to assemble the complete global sparse matrix before performing precisely this elimination? Static condensation allows us to avoid this step.

5.3 Static condensation

Consider one refined connection between two original graph vertices. The connection contains several finite element unknowns in its interior. These interior unknowns interact with each other along the connection and with the two endpoint vertices, but not directly with the interior unknowns of other connections. This local structure allows the interior unknowns of each connection to be eliminated independently. After this elimination, the effect of the whole refined connection can be represented only through its two endpoint vertices.

To formalize this idea, let E collect all interior unknowns of the connections and let V collect the 83 original vertex unknowns. Splitting Eq. (13) accordingly gives

$$\begin{pmatrix} K_{EE} & K_{EV} \\ K_{VE} & K_{VV} \end{pmatrix} \begin{pmatrix} c_E \\ c_V \end{pmatrix} = \begin{pmatrix} b_E \\ b_V \end{pmatrix}, \quad K_{EV} = K_{VE}^\top. \quad (14)$$

The block K_{EE} is block diagonal, with one tridiagonal block for each connection. The interior variables can therefore be eliminated before solving the global problem. At the algebraic level, this gives the Schur complement system

$$S c_V = g, \quad S = K_{VV} - K_{VE} K_{EE}^{-1} K_{EV}, \quad g = b_V - K_{VE} K_{EE}^{-1} b_E. \quad (15)$$

K_{ij} , its distance from the diagonal is $|i - j|$ and the bandwidth is defined as $\max_{K_{ij} \neq 0} |i - j|$. RCM reduces this maximum distance from 7989 to 937, meaning that the nonzero entries are packed much closer to the main diagonal. This reduction, however, does not necessarily imply a cheaper factorisation, since the amount of fill-in generated during the elimination also plays a crucial role.

The resulting system contains only the original graph vertices. Once c_V is known, the eliminated interior values are reconstructed by

$$c_E = K_{EE}^{-1}(b_E - K_{EV}c_V).$$

A direct implementation of Eq. (15) could first assemble the complete global matrix K^n and then compute its Schur complement. This would, however, preserve the expensive construction of the large sparse matrix that we want to avoid. Instead, we exploit the block structure already at the level of the individual connections.

For each connection, only its local finite element matrix and right-hand side are constructed. The local interior block is tridiagonal, so the interior unknowns can be eliminated efficiently using the Thomas algorithm. This local elimination produces a 2×2 Schur contribution involving only the two endpoint vertices of that connection, together with the corresponding contribution to the reduced right-hand side. These local 2×2 contributions are then assembled directly into the global Schur matrix. If several connections meet at the same graph vertex, their contributions are added to the same entries of the global system, in the same way as local finite element matrices are assembled into a global matrix. Therefore, a vertex with degree larger than one simply receives the sum of the contributions from all incident connections. Repeating this procedure over all connections directly produces the global Schur system on the 83 original vertices. In particular, the complete global matrix K^n is never assembled and the global Schur complement is never computed from it in one operation. Instead, the reduced 83×83 matrix is built directly from the locally condensed connection contributions. Its size therefore remains fixed independently of the number of finite elements used inside the connections.

The previous analysis showed that one time step of the original solver can be divided into three main phases. First, the complete global sparse matrix is constructed. Second, this matrix is factorized and finally, the resulting factors are used to solve the linear system. Initially, the factorization was the dominant cost; but resorting to a combination of natural ordering and LDL^T factorization substantially reduced it. After this optimisation, however, the construction of the complete global sparse matrix became the most expensive phase.

Static condensation changes this workflow. The new time step can be divided into four phases. First, the local finite element contributions are constructed and condensed independently on each connection. Second, the resulting 2×2 contributions are assembled into the global 83×83 Schur system. Third, this reduced system is factorised and then solved. Finally, the interior unknowns previously eliminated on each connection are recovered by back substitution⁴. The main computational advantage is therefore that the complete global sparse matrix is never assembled.

For the timing study, the same connectome is refined up to 128 elements per connection, corresponding to 143593 unknowns. The topology is kept fixed: only the finite element resolution along each connection is increased. This provides a controlled increase in the computational workload without modifying the underlying connectome.

Elements per connection	Unknowns	Full-system step [ms]	Condensed step [ms]	Ratio
8	7993	3.61	1.03	3.5
16	17033	8.34	1.99	4.2
32	35113	13.52	4.11	3.3
64	71273	35.02	6.73	5.2
128	143593	64.45	10.99	5.9

Table 8: Comparison between the complete full-system solver and the complete statically condensed solver. The full-system time includes the construction of the global sparse matrix, its LDL^T factorisation and the subsequent solve. The condensed time includes the local construction and elimination on all connections, the assembly and solution of the 83×83 Schur system and the final back substitution. For each mesh, the two solvers are executed consecutively in the same run so that their timings are measured under the same computational conditions.

Table 8 shows that static condensation is between 3.5 and 5.9 times faster than the optimized full-system solver over the tested refinements. At 128 elements per connection the full-system

⁴The 83×83 Schur matrix is stored as a dense matrix and factorised using a dense LDL^T factorization. Solving this system provides the values at the 83 original graph vertices. Once these vertex values are known, the interior finite element unknowns of every connection can be reconstructed independently using the local quantities computed during condensation.

step costs 64.45 ms, whereas the condensed formulation requires 10.99 ms. The timing breakdown also shows where most of the computational cost is spent. At the finest mesh, approximately 10.56 ms are spent processing the connections, 0.34 ms in back substitution and only 0.09 ms in the 83-vertex interface solve. The global part therefore represents less than one percent of the step, while almost all the remaining work consists of independent operations on the connections, a fact that demonstrates an opportunity for parallelism.

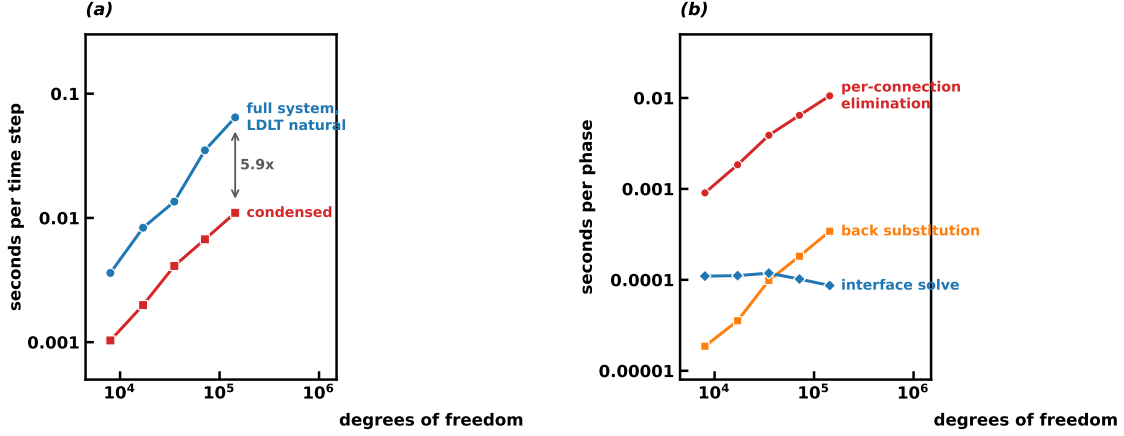


Figure 30: Static condensation. (a) median per-step cost of the condensed solver against the full-system reference, with the ratio at the finest mesh. (b) phase split of the condensed step: the interface solve is constant across refinement and the step is dominated by the independent per-connection eliminations.

At 64 elements per connection, the complete sequence of sequential optimisations reduces the cost from 140.3 ms per step for the original solver configuration to 6.73 ms for the condensed solver. This reduction is an *algorithmic* gain obtained with one thread. It will be kept separate from the parallel speedups measured in the following sections. Parallel programming techniques are discussed as the structure exposed by static condensation shows that different connections can be processed simultaneously.

5.4 Parallel implementation

We observe that the natural unit of parallel work is the single graph connection. Different connections can be processed simultaneously and once the interface solution is known, their interior values can also be reconstructed independently. We exploit this structure using both shared-memory and distributed-memory paradigms of parallelism.

5.4.1 OpenMP

In the OpenMP implementation all threads belong to the same process and share the same memory. The set of graph connections is divided among the available threads, so that each thread independently performs the local assembly and condensation of its assigned connections. The only shared quantity produced during this phase is the 83-vertex interface system. Different connections may contribute to the same vertex, so allowing all threads to write directly to the same matrix would create data races. Instead, every thread first accumulates its own local contribution to the Schur matrix and right-hand side. These thread-local contributions are merged once the local connection work is complete. A single thread then solves the small interface problem. Once the 83 vertex values are available, the back-substitution along the connections is again distributed among the threads.

5.4.2 MPI

The MPI implementation uses the same decomposition by connections, but the workers are now separate processes with separate memory spaces.

The connections are partitioned among MPI ranks so that each rank owns approximately the same amount of interior finite element work. Each rank processes its local connections independently

and constructs its local contribution to the 83-vertex Schur system.

Unlike OpenMP threads, MPI ranks cannot access each other’s memory. Consequently, the local interface contributions must be explicitly combined. For this reason, at each step, two `MPI_Allreduce` operations are performed: one for the sum of the local Schur matrices and the other for the global right-hand side assembly. After the reduction, every rank possesses the complete interface problem. Since this system is only 83×83 , each rank solves it independently rather than distributing such a small factorization. Each rank then reconstructs only the interior values of the connections it owns.

A hybrid implementation combines the two approaches: MPI partitions the graph connections among ranks, so that each rank is responsible for a local subgraph, while OpenMP threads parallelize the processing of the connections assigned to that rank.

5.5 Parallel performance

We now measure the OpenMP strong scaling of the condensed solver. In a strong-scaling experiment the numerical problem is kept fixed while the number of workers is increased. The connectome topology is therefore unchanged and every connection is discretised using 128 finite elements.

The experiments are performed on a processor with four physical cores and two hardware threads per core. Results up to four workers therefore correspond to at most one worker per physical core. The eight-worker configuration additionally uses simultaneous multithreading (SMT).

Threads	Per step [ms]	Speedup	Efficiency
sequential	9.58	–	–
2	5.65	1.69	0.85
4	3.37	2.84	0.71
8	2.06	4.64	0.58

Table 9: OpenMP strong scaling of the condensed step on the mesh with 128 elements per connection. Reported timings are representative per-step values over 200 time steps. Speedups are computed with respect to the sequential condensed reference measured in the same benchmark session, so absolute timings should not be compared directly with those reported in Table 8.

Table 9 shows a regular reduction of the execution time as threads are added. Two threads reduce the step from 9.58 to 5.65 ms, corresponding to a speedup of 1.69. With four threads, one per physical core, the step requires 3.37 ms, giving

$$S_4 = 2.84, \quad E_4 = \frac{S_4}{4} = 0.71.$$

The efficiency decreases at eight threads because the processor has only four physical cores, so the additional threads share the same hardware resources. At the same time, the serial and memory-bound parts of the computation become more significant as the parallel work is accelerated. The OpenMP code path itself introduces a small overhead: running it with a single thread costs 10.70 ms compared with the 9.58 ms sequential condensed reference. This overhead is reported separately and is not used as the baseline for the speedup.

Ranks	Per step [ms]	Reduction [ms]	Speedup	Efficiency
sequential	8.07	–	–	–
2	4.66	0.09	1.73	0.87
4	4.29	0.71	1.88	0.47
8	2.56	0.20	3.15	0.39

Table 10: MPI strong scaling of the condensed step on the same mesh. The reduction column is the median per-step cost of summing the interface system across the ranks through `MPI_Allreduce`. Each experiment measures its own sequential condensed reference in its own benchmark session, which is why the sequential row differs slightly from that of Table 9.

The MPI version shows the same basic behaviour at low worker counts. With two ranks the speedup is 1.73, very close to the two-thread OpenMP result. This is expected because the local numerical work performed on the connections is essentially the same in the two implementations and, with only two MPI ranks, the communication overhead introduced by the `MPI_Allreduce` remains small.

The difference between the two implementations appears in the presence of a synchronization step. OpenMP threads share memory and only need to merge their local accumulators, whereas MPI ranks must exchange their interface contributions through `MPI_Allreduce`. At four ranks, the measured reduction time is 0.71 ms per step and the total speedup is 1.88, while at eight ranks the reduction time decreases to 0.20 ms and the speedup reaches 3.15. A possible explanation is that, with more ranks, the amount of work assigned to each rank becomes smaller and more evenly balanced, so the ranks reach the collective operation at more similar times and spend less time waiting for one another. MPI introduces a synchronization and communication cost that the OpenMP implementation does not have on the same shared-memory node. This reflects the difference between the shared-memory parallelism used by OpenMP, where threads directly access a common address space and the distributed-memory parallelism used by MPI, where processes have separate memory spaces and must explicitly exchange data.

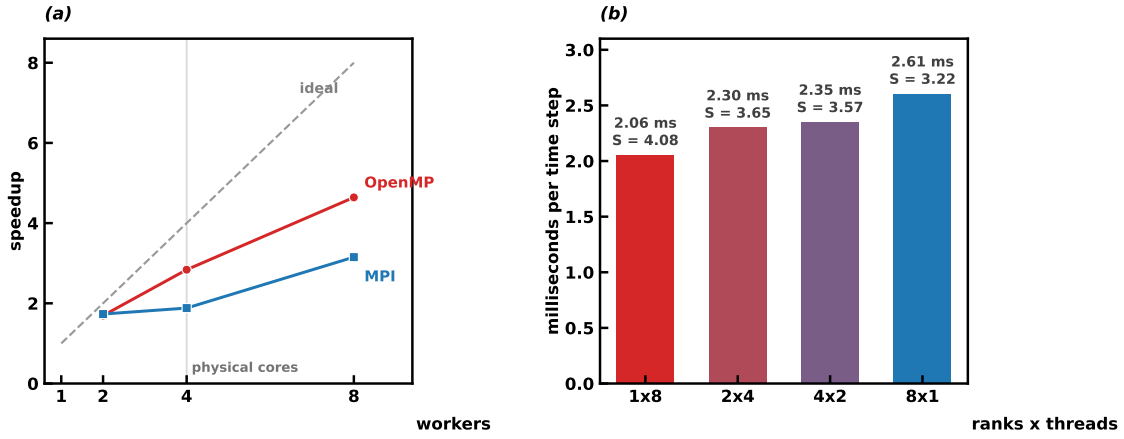


Figure 31: Parallel performance on the mesh with 128 elements per connection. (a) strong scaling of the OpenMP and MPI versions; each series is normalised by the sequential condensed reference of its own experiment. (b) the four arrangements of eight workers with the hybrid implementation, expressed as MPI ranks $\times$ OpenMP threads.

The hybrid experiment provides a final comparison at a fixed total of eight workers. The configurations range from one MPI rank with eight OpenMP threads to eight MPI ranks with one thread each. The measured step times are 2.06, 2.30, 2.35 and 2.61 ms for the 1×8 , 2×4 , 4×2 and 8×1 configurations, respectively.

The trend is monotonic: on this single node, replacing shared-memory threads with additional MPI ranks increases the synchronization cost without changing the amount of local numerical work. Pure OpenMP is therefore the most efficient configuration on the tested machine, while the hybrid configurations lie between the OpenMP and MPI extremes.

6 Conclusions and future work

This project developed a finite element framework for partial differential equations posed on metric graphs and applied it to a problem of current interest: the Fisher–Kolmogorov model of neurodegeneration on the brain connectome. The library is built around a compact core consisting of a graph topology with metric connections, a piecewise linear discretization along each connection and a common workflow for input handling, finite element assembly, solution and output. The benchmarks validated this framework on problems with known solutions: we treated stationary and time-dependent problems on star graphs, eigenvalue problems on other various topologies, like graphene-like structures and binary tree graphs, against the reference results of [6].

The study of Section 4 used the library as a laboratory for the Fisher–Kolmogorov equation on the 83-region connectome of [9]. Three findings stand out. First, at one element per connection the metric-graph formulation reproduces the behaviour of the nodal network models while remaining computationally inexpensive and refinement inside the connections remains available. Second, the balance between reaction and transport, summarised by the Damköhler number, governs both the physical regime and the numerical robustness of the model and the lumped mass matrix is the appropriate choice, as a stabilization technique, in the reaction-dominated regime. Third, connectivity alone orders the four lobes only partially: with the regionally heterogeneous reaction rates of [8] transferred to our connectome, we retrieve the clinical progression of the disease.

The performance analysis of Section 5 showed that the unexpected cost of the sequential baseline was due to the fill-in generated by the default matrix ordering during the factorization stage. The natural numbering of the unknowns, with connection interiors first and graph vertices last, exposed a structure that was exploited through static condensation. By eliminating the interior unknowns locally, the global problem was reduced to the 83 original graph vertices. On a four-core laptop, combining this reduction with parallel execution over the interior nodes reduced significantly the per-step cost within the time loop of the finest connectome discretization.

Several directions remain open.

- The MPI version was measured only on a single shared-memory node, where the OpenMP implementation is preferable. Since the main purpose of MPI is to enable distributed-memory execution, a multi-node scalability study is the natural next step.
- A per-step cost of about two milliseconds at 143593 unknowns makes many-run studies practical; parameter calibration and the uncertainty quantification of [8] are the settings in which the speedup would be most valuable.
- The data pipeline already reconstructs the fine graph with 1015 nodes from which the 83 regions are aggregated; repeating the study at the finer parcellation would test the robustness of the staging results.
- More advanced disease models from the literature, including multiple protein species and patient-specific parameters, as well as more complex brain-graph domains, could also be incorporated within this same computational framework.

The quantitative results reported throughout this work are backed by stored and regenerable benchmark records, as described in the next section.

7 C++ implementation

7.1 Repository reference

The project is delivered as a single repository organized as follows:

<code>FEMG/</code>	the library: headers and sources of the problem classes
<code>test_problem/</code>	one driver per experiment family (parabolic, spectral, fisher_kolmogorov, performance)
<code>benchmarks/</code>	one numbered record per experiment of the report, each with a README, the commands used and the stored results
<code>scripts/</code>	benchmark runners, figure generation and verification
<code>data/</code>	graph files and the connectome preparation pipeline
<code>verification/</code>	internal correctness checks of the solver
<code>report/</code>	this document

The records are the organising principle: every figure and every number of the report is produced from files stored under `benchmarks/` and each record documents how to regenerate them.

7.2 Dependencies

The library is written in C++17 and built with CMake. It depends on Eigen for sparse and dense linear algebra and on the Boost Graph Library for the graph topology management. OpenMP and MPI are optional dependencies: when available, CMake enables the parallel performance executables, while the core library can be built and used without either of them. Post-processing uses Python 3 with NumPy, Matplotlib and pandas. The default build type is Release with `-O2`.

7.3 Computational approach to differential problems

All problems share the abstract base class `quantum_graph_problem`. The topology is a Boost undirected adjacency list whose vertex property carries the coordinates used for visualisation and whose edge property carries the metric data of the connection, its length and its number of finite element cells. `init()` reads the graph file and builds the numbering of the unknowns, interior nodes of each connection first and the original vertices last. The base class stores the shared discrete objects, the discrete Hamiltonian H , the mass matrix M and the solution vector and prescribes the workflow as four virtual steps: `set_coefficients()`, `assembly()`, `solve()` and `sol_export()`. Three concrete classes derive from it: `parabolic_problem` implements the θ -method for the heat equation with Dirichlet conditions on selected vertices, while the source term and the initial condition are provided as callable functions. The elliptic benchmarks are stationary solutions computed through this parabolic pipeline. `spectral_problem` solves the generalised eigenvalue problem $Hx = \lambda Mx$ for a requested number of modes and also exposes the eigenvalues of the combinatorial Laplacian. `fisher_kolmogorov_problem` implements the two time schemes: an implicit scheme with Backward Euler and a damped and safeguarded Newton iteration and the semi-implicit scheme of [8], with one diffusion coefficient per connection, one reaction rate per region and optional mass lumping as a stabilization technique. The condensed solver is deliberately kept outside the library, as a header-only engine under `test_problem/performance/` that replays the semi-implicit step without modifying the validated classes.

7.4 Input data

Graphs are plain text files. The header carries the number of vertices and connections; an optional block with two coordinates per vertex follows and each connection is described by its two endpoint indices, its length and its number of cells. The same reader accepts both the coordinate-free and the coordinate-carrying variant. The `data/` directory provides the analytical geometries treated in this work: the interval, the four-pointed star, the graphene-like graph and the binary tree, at several resolutions. The connectome enters through a documented preparation pipeline: the public files of the Budapest Reference Connectome [10] are reduced to the 83-region connectome of [9] by the scripts under `scripts/`, which also emit the per-connection weights and the refined meshes, from one to 128 elements per connection. Every derived file is regenerable from the raw download.

7.5 Setting coefficients

Physical parameters travel through constructors and setters. The parabolic constructor takes the diffusion coefficient, the final time, the time step and θ ; sources, initial data and vertex Dirichlet data are function objects evaluated on the metric coordinate of each connection. The Fisher–Kolmogorov class accepts one diffusion coefficient per connection, such as the normalised fibre weights, one reaction rate and one initial value per region; further setters select the time scheme, the stabilization technique, the Newton parameters and the frequency at which the solution is exported. After the setters, `set_coefficients()` validates and stores everything the assembly needs.

7.6 Assembly procedures

`assembly()` loops over the connections and, on each connection, over its cells. The elemental blocks of the piecewise linear basis on a cell of size h are accumulated as triplets and assembled into compressed Eigen sparse matrices: the consistent mass block, its lumped variant when mass lumping is selected and the stiffness block scaled by the diffusion coefficient of the connection. The nonlinear reaction of the Fisher–Kolmogorov model is integrated by Gauss quadrature applied to the nodal interpolants of the rate and of the state, with a vertex rule consistent with the lumped mass when lumping is on. The Kirchhoff conditions at the vertices need no explicit treatment, since they arise naturally from the weak form when the test functions are continuous across a vertex; Dirichlet data are imposed algebraically on the selected vertices.

7.7 Solvers

The linear heat equation factorises its θ -method matrix once, before the time loop, with the sparse LU of Eigen and reuses the factorisation at every step. The Fisher–Kolmogorov schemes refactorise: the semi-implicit scheme at every step, because the extrapolated reaction changes the matrix and the fully implicit scheme at every iteration of the damped and safeguarded Newton method. The eigenvalue problems are small enough for a dense generalised symmetric eigensolver. The performance study adds the measured alternatives: the symmetric LDL^T factorisation on the natural ordering, the per-connection static condensation with its dense 83×83 interface and the OpenMP, MPI and hybrid variants of the condensed step.

7.8 Exporting results

`sol_export()` writes ParaView files. Time-dependent problems produce a `solution.pvd` collection referencing one `solution_XXXX.vtp` polyline file per stored step, with a configurable stride to bound the output volume; the spectral problems write the domain and one file per eigenfunction, together with a CSV of the eigenvalues. The drivers add their own CSV summaries, timings, staging times and regional means, which are the inputs of the figures.

7.9 Result postprocessing

Post-processing is Python. Each figure of the report is produced by one script under `scripts/`. `scripts/run-benchmark.sh` automates the execution of a benchmark: it rebuilds the corresponding executable, runs the experiment and updates the stored results. Each benchmark directory also contains a README describing the experiment and how its results should be interpreted.

7.10 Example of code usage

The driver of Test E3 shows the complete arc of the interface: construction with the physical and time parameters, data supplied as function objects, initialisation from a graph file, coefficient evaluation, assembly, solution and the error check against the exact solution.

```
#include "parabolic_problem.hpp"
#include "star_sine_functions.hpp"
#include <iostream>
#include <memory>
#include <string>

int main(int argc, char *argv[]) {
```

```

const double diffusion = 1.0;
const double T = 1.0;
const double deltat = 0.005;
const double theta = 1.0;

std::string graph_file = "data/star_4.txt";
char *local_argv[] = {argv[0], graph_file.data()};

femg::parabolic_problem problem(diffusion, T, deltat, theta);
problem.set_initial_condition(std::make_shared<StarSineInitialCondition>());
problem.set_source_function(std::make_shared<StarSineForcingTerm>());
problem.set_dirichlet_vertices(
    {0, 1, 2, 3, 4},
    [](std::size_t, double) {
        return 0.0;
    });
problem.set_output_directory("output/visualization/star_sine");

if (argc >= 2) {
    problem.init(argc, argv);
} else {
    problem.init(2, local_argv);
}

problem.set_coefficients();
problem.assemble_matrices();
problem.print_matrix_summary(std::cout);
problem.solve();

const StarSineExactSolution exact_solution;
const double error_l2 = problem.compute_l2_error(exact_solution);
std::cout << "Final L2 error against u = sin(2*pi*x): "
            << error_l2 << "\n";

return 0;
}

```

Building and running the example requires three commands:

```

cmake -S . -B build
cmake --build build
./build/test_heat_star_sine

```

The resulting solution is written to `output/visualization/star_sine/solution.pvd` and can be visualised with ParaView. The same sequence of operations provides the basic interface for defining and running new experiments with the library.

7.11 Parallel code excerpts

In the OpenMP implementation, the graph connections are divided among the threads. Each thread processes its assigned connections independently and accumulates the corresponding Schur matrix and right-hand side in private local storage. Once the local work is completed, the thread-local contributions are combined to form the global interface system.

```

void condense_omp(const std::vector<double> &extrapolated,
const std::vector<double> &state) {
    schur_.setZero();
    condensed_rhs_.setZero();
    const auto n_edges = static_cast<std::ptrdiff_t>(edges_.size());
    #pragma omp parallel
    {
        Eigen::MatrixXd schur_local =
            Eigen::MatrixXd::Zero(schur_.rows(), schur_.cols());
        Eigen::VectorXd rhs_local =
            Eigen::VectorXd::Zero(condensed_rhs_.size());
        #pragma omp for schedule(static)
        for (std::ptrdiff_t e = 0; e < n_edges; ++e) {
            process_edge(edges_[static_cast<std::size_t>(e)],
                extrapolated, state, schur_local, rhs_local);
        }
        #pragma omp critical
    }
}

```

```

        {
            schur_ += schur_local;
            condensed_rhs_ += rhs_local;
        }
    }
}

```

In the MPI implementation, each rank processes only the graph connections assigned to it and computes its local contributions to the Schur matrix and right-hand side. Two `MPI_Allreduce` operations then sum these local contributions and make the resulting global interface system available on every rank. Omitting the instructions used only for performance measurements, the essential part of the distributed time step is:

```

stepper.condense(extrapolated, state);
MPI_Allreduce(MPI_IN_PLACE, stepper.schur().data(),
    static_cast<int>(n_vertices * n_vertices), MPI_DOUBLE,
    MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(MPI_IN_PLACE, stepper.condensed_rhs().data(),
    static_cast<int>(n_vertices), MPI_DOUBLE, MPI_SUM,
    MPI_COMM_WORLD);
const Eigen::VectorXd vertices = stepper.solve_interface();
stepper.scatter_vertices(vertices, state);
stepper.back_substitute(vertices, state);

```

After the reduction, no further communication is required: every rank solves the same 83×83 system and reconstructs the interior values of its own connections.

A Properties of the Combinatorial Laplacian

The combinatorial Laplacian $L_{\mathcal{G}} = MM^{\top}$ satisfies the following properties:

- Symmetry

$$L_{\mathcal{G}}^{\top} = (MM^{\top})^{\top} = (M^{\top})^{\top}M^{\top} = MM^{\top} = L_{\mathcal{G}}.$$

- Positive semidefiniteness

$$x^{\top}L_{\mathcal{G}}x = x^{\top}MM^{\top}x = (M^{\top}x)^{\top}(M^{\top}x) = \|M^{\top}x\|_2^2 \geq 0,$$

hence $x^{\top}L_{\mathcal{G}}x \geq 0$ for all $x \in \mathbb{R}^V$.

- Singularity

$$(M^{\top}f)_e = f(v_i) - f(v_j) \quad \forall e = \{v_i, v_j\} \in \mathcal{E}, f \in \mathbb{R}^V$$

So $(M^{\top}f)_e = 0$ if and only if $f(v_i) = f(v_j)$ that is, f must be constant on every pair of adjacent vertices. If $\mathcal{G}$ is connected, this forces f to be globally constant, so

$$\ker(M^{\top}) = \text{span}\{(1, 1, \dots, 1)^{\top}\}.$$

Since $L_{\mathcal{G}} = MM^{\top}$, we have $L_{\mathcal{G}}f = 0 \iff M^{\top}f = 0$, hence

$$\ker(L_{\mathcal{G}}) = \ker(M^{\top}) = \text{span}\{(1, 1, \dots, 1)^{\top}\},$$

confirming that $L_{\mathcal{G}}$ is singular. More generally, the constant function argument applies independently on each connected component: a vector f satisfying $M^{\top}f = 0$ must be constant within each component but may take different values across components. Therefore

$$\dim \ker(L_{\mathcal{G}}) = k,$$

where k is the number of connected components of $\mathcal{G}$ ⁵.

- Meaning

$$(Mg)_i = \sum_{e \ni v_i} \pm g_e, \forall g \in \mathbb{R}^E$$

where the sign is +1 if the flux along e is outgoing from v_i and -1 if it is incoming to v_i , reflecting the local flow configuration at its incident edges. This is the discrete analogue of a divergence: M accumulates the signed flux of g at each node.

Dually, for $f \in \mathbb{R}^V$, the action of $M^{\top}$ on edge $e = \{v_i, v_j\}$ reads

$$(M^{\top}f)_e = f(v_i) - f(v_j),$$

which is the difference of the nodal values at the endpoints of e , i.e. the discrete analogue of a derivative. Thus $M^{\top}$ plays the role of a *discrete gradient*.

Since M is a discrete divergence and $M^{\top}$ a discrete gradient, their composition $L_{\mathcal{G}} = MM^{\top}$ is naturally interpreted as a *discrete Laplacian*, which motivates calling $L_{\mathcal{G}}$ the Laplacian of the combinatorial graph $\mathcal{G}$.

Input graphs are specified in a simple `.graph` plain-text format: the first line gives $|\mathcal{V}|$ and $|\mathcal{E}|$, followed by one line per edge with endpoint indices and length.

⁵In a (possibly disconnected) graph, a connected component is defined as a maximal connected subgraph, i.e. a largest connected subset of vertices (together with the edges between them).

B Derivation of the weak and algebraic formulations

B.1 Problem formulation

A metric graph $\Gamma = (\mathcal{V}, \mathcal{E})$ consists of a finite set of vertices and a finite set of edges. The vertex set is generally partitioned into three disjoint subsets:

$$\mathcal{V} = \mathcal{T} \cup \mathcal{V}_1 \cup \mathcal{V}_2,$$

where $\mathcal{T}$ collects the vertices of degree one (i.e. they are the terminal points of the graph), $\mathcal{V}_1$ collects the *simple junction* vertices, at which the flux is conserved and $\mathcal{V}_2$ collects the *generalized junction* vertices at which a different local dynamics can be prescribed. On Γ we consider a system of parabolic problems. Each line of this system lives within an edge e and communicates with the others through boundary conditions.

For each edge $e \in \mathcal{E}$ and $t \in (0, T]$, the unknown $u_e(s, t) = u(s_e, t)$ satisfies

$$\frac{\partial u_e}{\partial t} - \frac{\partial}{\partial s} \left(\mu_e \frac{\partial u_e}{\partial s} \right) + r_e u_e = f_e, \quad 0 < s_e < \ell_e, \quad \forall e \in \mathcal{E}, \quad (16)$$

where $\mu_e = \mu_e(s, t) > 0$ is the diffusion coefficient, $r_e = r_e(s, t) \geq 0$ is the reaction coefficient and $f_e = f_e(s, t)$ is a given source term on e . The initial condition on each edge is the following

$$u_e(s, 0) = u_e^0(s), \quad 0 < s < \ell_e, \quad \forall e \in \mathcal{E}.$$

At each terminal vertex, either a Dirichlet boundary condition or a homogeneous Neumann condition can be imposed. At each simple junction vertex $v_p \in \mathcal{V}_1$, the solution is continuous and the total flux is conserved:

$$\sum_{e \in \mathcal{E}_{v_p}} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} \right]_{v_p} = 0, \quad \forall v_p \in \mathcal{V}_1, \quad (17)$$

where $\mathcal{E}_{v_p}$ denotes the set of edges incident to v_p and $[\partial/\partial \mathbf{n}_e]_{v_p}$ denotes the outward normal derivative along e with respect to v_p . At each generalized junction vertex $v_q \in \mathcal{V}_2$, a local variable $u_q = u_q(t)$ is associated with the vertex and satisfies

$$c_q \frac{du_q}{dt} + r_q u_q = \sum_{e \in \mathcal{E}_{v_q}} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} \right]_{v_q} + f_q, \quad \forall v_q \in \mathcal{V}_2, \quad (18)$$

where $c_q > 0$ is a relaxation coefficient, $f_q = f_q(t)$ is a local source term, and $[\partial/\partial \mathbf{n}_e]_{v_q}$ is the outward normal derivative along each incident edge with respect to v_q . These are two different versions of the so-called Kirchhoff conditions.

B.2 Weak formulation

We consider the parabolic problem (16) on the metric graph $\Gamma = (\mathcal{V}, \mathcal{E})$, together with the Kirchhoff conditions (17)–(18), homogeneous Neumann boundary conditions at the terminal vertices and the global continuity constraints. The spatial differential operator acting on each edge is

$$\mathcal{L}_e u_e = - \frac{\partial}{\partial s} \left(\mu_e \frac{\partial u_e}{\partial s} \right) + r_e u_e, \quad (19)$$

so that (16) reads $\partial_t u_e + \mathcal{L}_e u_e = f_e$ on each $e \in \mathcal{E}$. Note that when μ_e is constant and $r_e = V_e(s)$ is a potential, $\mathcal{L}_e$ reduces to the quantum-graph Hamiltonian $\mathcal{H}_e u_e = -\mu_e \frac{\partial^2 u_e}{\partial s^2} + V_e u_e$. The weak formulation is obtained by multiplying (16) by a test function $w \in H^1(\Gamma)$, integrating over each edge, summing over $\mathcal{E}$ and integrating by parts the diffusion term. At each junction vertex, the boundary terms arising from integration by parts are replaced using the Kirchhoff conditions (17)–(18), which cancel for $v_p \in \mathcal{V}_1$ and contribute a local ODE term for $v_q \in \mathcal{V}_2$. Also contributions coming from terminal vertices cancel, as we consider homogeneous Neumann conditions. The weak problem reads:

Find $u(s, t) \in H^1(\Gamma)$ such that

$$\sum_{e \in \mathcal{E}} \int_e \left[\frac{\partial u_e}{\partial t} w_e \right] ds + a(u, w) - \sum_{v_q \in \mathcal{V}_2} c_q \frac{du_q}{dt} w_q = F(w) \quad \forall w \in H^1(\Gamma), \quad (20)$$

where the bilinear form $a : H^1(\Gamma) \times H^1(\Gamma) \rightarrow \mathbb{R}$ is given by

$$a(u, w) = \sum_{e \in \mathcal{E}} \int_e \left[\mu_e \frac{\partial u_e}{\partial s} \frac{\partial w_e}{\partial s} + r_e u_e w_e \right] ds - \sum_{v_q \in \mathcal{V}_2} r_q u_q w_q, \quad (21)$$

and the linear functional $F : H^1(\Gamma) \rightarrow \mathbb{R}$ is

$$F(w) = \sum_{e \in \mathcal{E}} \int_e f_e w_e ds - \sum_{v_q \in \mathcal{V}_2} f_q w_q. \quad (22)$$

B.3 Derivation of the variational formulation

We start from the strong problem on each edge $e \in \mathcal{E}$:

$$\frac{\partial u_e}{\partial t} - \frac{\partial}{\partial s} \left(\mu_e \frac{\partial u_e}{\partial s} \right) + r_e u_e = f_e, \quad s \in (0, \ell_e), \quad t \in (0, T], \quad (B.1)$$

supplemented by the Kirchhoff conditions (17)–(18), the Neumann constraints at the terminal vertices and the global continuity requirements.

Let $V = H^1(\Gamma)$ be the space of functions $w : \Gamma \rightarrow \mathbb{R}$ such that $w_e \in H^1(0, \ell_e)$ for each $e \in \mathcal{E}$ and w is continuous at every junction vertex. We multiply (B.1) by a test function $w \in V$, integrate over e and sum over all edges:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_e}{\partial t} w ds - \sum_{e \in \mathcal{E}} \int_e \frac{\partial}{\partial s} \left(\mu_e \frac{\partial u_e}{\partial s} \right) w ds + \sum_{e \in \mathcal{E}} \int_e r_e u_e w ds = \sum_{e \in \mathcal{E}} \int_e f_e w ds. \quad (B.2)$$

We integrate by parts the diffusion term on each edge:

$$- \int_e \frac{\partial}{\partial s} \left(\mu_e \frac{\partial u_e}{\partial s} \right) w ds = \int_e \mu_e \frac{\partial u_e}{\partial s} \frac{\partial w}{\partial s} ds - \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} w \right]_{\partial e}, \quad (B.3)$$

where the boundary term is evaluated at the two endpoints of e with outward normal sign. Summing over all edges and grouping boundary terms by vertex:

$$\sum_{e \in \mathcal{E}} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} w \right]_{\partial e} = \sum_{v \in \mathcal{V}} w(v) \sum_{e \in \mathcal{E}_v} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} \right]_v. \quad (B.4)$$

We now apply the boundary and junction conditions at each vertex:

- At terminal vertices $v \in \mathcal{T}$: if homogeneous Neumann, $[\mu_e \partial_{\mathbf{n}_e} u_e]_v = 0$ and the contribution vanishes. If Dirichlet, $w(v) = 0$ for test functions in the appropriate subspace and again the contribution vanishes.
- At simple junction vertices, the Kirchhoff condition (17) gives $\sum_{e \in \mathcal{E}_{v_p}} [\mu_e \partial_{\mathbf{n}_e} u_e]_{v_p} = 0$, so the contribution vanishes.
- At generalized junction vertices $v_q \in \mathcal{V}_2$: the Kirchhoff condition (18) gives

$$\sum_{e \in \mathcal{E}_{v_q}} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} \right]_{v_q} = c_q \frac{du_q}{dt} + r_q u_q - f_q.$$

Therefore (B.4) becomes:

$$\sum_{e \in \mathcal{E}} \left[\mu_e \frac{\partial u_e}{\partial \mathbf{n}_e} w \right]_{\partial e} = \sum_{v_q \in \mathcal{V}_2} w_q \left(c_q \frac{du_q}{dt} + r_q u_q - f_q \right). \quad (B.5)$$

Substituting (B.3) and (B.5) into (B.2) and rearranging:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_e}{\partial t} w \, ds + a(u, w) - \sum_{v_q \in \mathcal{V}_2} c_q \frac{du_q}{dt} w_q = F(w) \quad \forall w \in V, \quad (\text{B.6})$$

where

$$a(u, w) = \sum_{e \in \mathcal{E}} \int_e \left[\mu_e \frac{\partial u_e}{\partial s} \frac{\partial w_e}{\partial s} + r_e u_e w_e \right] ds - \sum_{v_q \in \mathcal{V}_2} r_q u_q w_q,$$

$$F(w) = \sum_{e \in \mathcal{E}} \int_e f_e w \, ds - \sum_{v_q \in \mathcal{V}_2} f_q w_q.$$

This is the weak formulation (20).

B.4 Galerkin formulation

Let $V_h \subset V$ be the finite-dimensional subspace spanned by the basis functions $\{\phi_j^e\}_{e \in \mathcal{E}, j=1, \dots, n_e-1}$ (interior node hat functions on each edge) and $\{\phi_v\}_{v \in \mathcal{V}}$ (vertex basis functions), satisfying $\phi_j^e(s_k^{e'}) = \delta_{ee'} \delta_{jk}$ and $\phi_v(v') = \delta_{vv'}$. The discrete solution is written as

$$u_h(s, t) = \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \alpha_j^e(t) \phi_j^e(s) + \sum_{v \in \mathcal{V}} \alpha_v(t) \phi_v(s). \quad (\text{B.7})$$

Substituting (B.7) into (B.6) and testing on each basis function, we obtain two families of equations.

Testing on $\phi_k^{e'}$, interior node of edge e' . Since $\phi_k^{e'}$ has support only on e' and vanishes at all vertices, the $\mathcal{V}_2$ term in (B.6) does not contribute. The time term gives:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_h}{\partial t} \phi_k^{e'} \, ds = \sum_{j=1}^{n_{e'}-1} \dot{\alpha}_j^{e'} \underbrace{\int_{e'} \phi_j^{e'} \phi_k^{e'} \, ds}_{P_{kj}^{e'}} + \sum_{v \in \mathcal{V}} \dot{\alpha}_v \underbrace{\int_{e'} \phi_v \phi_k^{e'} \, ds}_{R_{kv}^{e'}},$$

where only the $e = e'$ term survives due to the local support of $\phi_k^{e'}$ and $R_{kv}^{e'}$ is nonzero only for the (at most two) vertices incident to e' . The stiffness term gives:

$$a(u_h, \phi_k^{e'}) = \sum_{j=1}^{n_{e'}-1} \alpha_j^{e'} \underbrace{\int_{e'} \left[\mu_{e'} \frac{\partial \phi_j^{e'}}{\partial s} \frac{\partial \phi_k^{e'}}{\partial s} + r_{e'} \phi_j^{e'} \phi_k^{e'} \right] ds}_{A_{kj}^{e'}} + \sum_{v \in \mathcal{V}} \alpha_v \underbrace{\int_{e'} \left[\mu_{e'} \frac{\partial \phi_v}{\partial s} \frac{\partial \phi_k^{e'}}{\partial s} + r_{e'} \phi_v \phi_k^{e'} \right] ds}_{B_{kv}^{e'}}.$$

Testing on $\phi_{v'}$, vertex basis function. Since $\phi_{v'}(v_q) = \delta_{v'v_q}$, the $\mathcal{V}_2$ time term in (B.6) contributes $-c_{v'} \dot{\alpha}_{v'}$ if $v' \in \mathcal{V}_2$ and zero otherwise. The remaining time term gives:

$$\sum_{e \in \mathcal{E}} \int_e \frac{\partial u_h}{\partial t} \phi_{v'} \, ds = \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \dot{\alpha}_j^e \underbrace{\int_e \phi_j^e \phi_{v'} \, ds}_{R_{v'j}^e} + \sum_{v \in \mathcal{V}} \dot{\alpha}_v \underbrace{\sum_{e \in \mathcal{E}} \int_e \phi_v \phi_{v'} \, ds}_{Q_{vv'}},$$

where $Q_{vv'}$ is nonzero only when v and v' share at least one edge. The stiffness term gives:

$$a(u_h, \phi_{v'}) = \sum_{e \in \mathcal{E}} \sum_{j=1}^{n_e-1} \alpha_j^e \underbrace{\int_e \left[\mu_e \frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + r_e \phi_j^e \phi_{v'} \right] ds}_{B_{v'j}^e} + \sum_{v \in \mathcal{V}} \alpha_v \underbrace{\left[\sum_{e \in \mathcal{E}} \int_e \left(\mu_e \frac{\partial \phi_v}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + r_e \phi_v \phi_{v'} \right) ds - r_v \delta_{vv'} \cdot \mathbf{1}_{[v \in \mathcal{V}_2]} \right]}_{G_{vv'}},$$

where $\mathbf{1}_{[v \in \mathcal{V}_2]}$ denotes the indicator function of the set $\mathcal{V}_2$, equal to 1 if v is a junction of the second type and 0 otherwise.

B.5 Matrix formulation

Collecting all unknowns into $\mathbf{u} = [\mathbf{u}_{\mathcal{E}}, \mathbf{u}_{\mathcal{V}}]^\top$, where $\mathbf{u}_{\mathcal{E}} = \{\alpha_j^e\}$ and $\mathbf{u}_{\mathcal{V}} = \{\alpha_v\}$, the Galerkin system is

$$\mathbf{M} \dot{\mathbf{u}}(t) + \mathbf{L} \mathbf{u}(t) = \mathbf{f}(t), \quad (\text{B.8})$$

with mass matrix and stiffness matrix

$$\mathbf{M} = \begin{bmatrix} \mathbf{P} & \mathbf{R}^\top \\ \mathbf{R} & \mathbf{Q} + \mathbf{C} \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{G} \end{bmatrix}, \quad (\text{B.9})$$

where the blocks are defined as follows:

- $P_{kj}^e = \int_e \phi_j^e \phi_k^e \, ds;$
- $R_{kv}^e = \int_e \phi_v \phi_k^e \, ds;$
- $Q_{v'v} = \sum_{e \in \mathcal{E}} \int_e \phi_v \phi_{v'} \, ds;$
- $C_{qq} = -c_q$ for $v_q \in \mathcal{V}_2$, zero elsewhere;
- $A_{kj}^e = \int_e \left[\mu_e \frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_k^e}{\partial s} + r_e \phi_j^e \phi_k^e \right] ds;$
- $B_{vj}^e = \int_e \left[\mu_e \frac{\partial \phi_j^e}{\partial s} \frac{\partial \phi_v}{\partial s} + r_e \phi_j^e \phi_v \right] ds;$
- $G_{v'v} = \sum_{e \in \mathcal{E}} \int_e \left[\mu_e \frac{\partial \phi_v}{\partial s} \frac{\partial \phi_{v'}}{\partial s} + r_e \phi_v \phi_{v'} \right] ds - r_v \delta_{vv'} \cdot \mathbf{1}_{[v \in \mathcal{V}_2]}.$

B.6 Fully discrete system

Discretizing (B.8) in time by the θ -method with uniform step $\Delta t = T/N_T$ and setting $\mathbf{u}^n \approx \mathbf{u}(t^n)$:

$$\mathbf{M} \frac{\mathbf{u}^n - \mathbf{u}^{n-1}}{\Delta t} + \mathbf{L} (\theta \mathbf{u}^n + (1 - \theta) \mathbf{u}^{n-1}) = \theta \mathbf{f}^n + (1 - \theta) \mathbf{f}^{n-1},$$

which rearranges to

$$\mathbf{H}_\theta \mathbf{u}^n = (\mathbf{M} - (1 - \theta) \Delta t \mathbf{L}) \mathbf{u}^{n-1} + \Delta t (\theta \mathbf{f}^n + (1 - \theta) \mathbf{f}^{n-1}), \quad (\text{B.10})$$

where $\mathbf{H}_\theta = \mathbf{M} + \theta \Delta t \mathbf{L}$. The choice $\theta = 1$ recovers the implicit Euler method, while $\theta = 1/2$ gives the Crank-Nicolson scheme. Explicitly:

$$\mathbf{H}_\theta = \begin{bmatrix} \mathbf{P} + \theta \Delta t \mathbf{A} & \mathbf{R}^\top + \theta \Delta t \mathbf{B} \\ \mathbf{R} + \theta \Delta t \mathbf{B}^\top & \mathbf{Q} + \mathbf{C} + \theta \Delta t \mathbf{G} \end{bmatrix}. \quad (\text{B.11})$$

Since $a(\cdot, \cdot)$ is symmetric and coercive on V and $\mathbf{M}$ is symmetric positive definite, $\mathbf{H}_\theta$ is symmetric positive definite for any $\Delta t > 0$ and $\theta \in (0, 1]$ and the algebraic formulation of the parabolic case of partial differential equations on graphs admits a unique solution at each time step.

References

- [1] Peter Kuchment. *Quantum Graphs: An Introduction and a Brief Survey*. 2008. arXiv: 0802.3442 [math-ph]. URL: <https://arxiv.org/pdf/0802.3442>.
- [2] Gregory Berkolaiko and Peter Kuchment. *Introduction to Quantum Graphs*. Vol. 186. Mathematical Surveys and Monographs. American Mathematical Society, 2013.
- [3] Michele Benzi. *Numerical Analysis of Quantum Graphs*. Plenary talk slides, Householder Symposium XIX. Accessed: August 30, 2026. Spa, Belgium, June 2014. URL: <https://sites.uclouvain.be/HHXIX/Plenaries/Benzi.pdf>.
- [4] R. Ciegis and N. Tumanova. “Finite-Difference Schemes for Parabolic Problems on Graphs”. In: *Lithuanian Mathematical Journal* 50.2 (2010), pp. 164–178.
- [5] Stefano Abbate and Andrea Di Primio. *Finite Element Methods on Graphs*. Project Report. MSc in Computational Science and Engineering, supervisor: Prof. P. Zunino. Politecnico di Milano, 2020.
- [6] Mario Arioli and Michele Benzi. *A Finite Element Method for Quantum Graphs*. Technical Report TR/PA/17/54. CERFACS, 2017. URL: <https://cerfacs.fr/wp-content/uploads/2017/03/Cerfacs-TR-17-54-1.pdf>.
- [7] Johannes Weickenmeier et al. “A Physics-Based Model Explains the Prion-Like Features of Neurodegeneration in Alzheimer’s Disease, Parkinson’s Disease, and Amyotrophic Lateral Sclerosis”. In: *Journal of the Mechanics and Physics of Solids* 124 (2019), pp. 264–281. DOI: 10.1016/j.jmps.2018.10.013.
- [8] Mattia Corti et al. “Uncertainty Quantification for Fisher–Kolmogorov Equation on Graphs with Application to Patient-Specific Alzheimer’s Disease”. In: *ESAIM: Mathematical Modelling and Numerical Analysis* 58.6 (2024), pp. 2135–2154. DOI: 10.1051/m2an/2023095.
- [9] Sveva Fornari et al. “Prion-like spreading of Alzheimer’s disease within the brain’s connectome”. In: *Journal of the Royal Society Interface* 16.159 (2019), p. 20190356. DOI: 10.1098/rsif.2019.0356.
- [10] Balázs Szalkai et al. “Parameterizable consensus connectomes from the Human Connectome Project: the Budapest Reference Connectome Server v3.0”. In: *Cognitive Neurodynamics* 11 (2017), pp. 113–116. DOI: 10.1007/s11571-016-9407-z.